



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Specifica Tecnica_G

Versione	0.2.2
Stato	In corso
Redazione	Tutto il gruppo
Verifica_G	Da verificare
Approvazione	Da approvare
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica
0.2.2	26/04/2026	Giulia Barzon	Aggiornati diagrammi frontend	Joseph Grant
0.2.1	25/04/2026	Giulia Barzon	Aggiornamento requisiti soddisfatti	Joseph Grant
0.2.0	25/04/2026	Elisa Marchioro	Aggiornamento subsubsection caching	Joseph Grant
0.1.9	24/04/2026	Matteo Cuogo	Aggiornamento dello stato di soddisfacimento di alcuni requisiti	Giulia Barzon
0.1.8	24/04/2026	Tommaso Tombacco	Modifica sezione AiWidget e modifica dello stato dei requisiti legati ad AI	Giulia Barzon
0.1.7	23/04/2026	Giulia Barzon	Aggiunte componenti AI, migliorata sezione 3	
0.1.6	23/04/2026	Elisa Marchioro	Sistemata formattazione, aggiunta sezione API _G e aggiunta diagrammi UML backend	Chiara Grossele
0.1.5	23/04/2026	Giulia Barzon	Rivista sezione layered architecture	Elisa Marchioro
0.1.4	22/04/2026	Giulia Barzon	Rivista sezione backend e strategy	Elisa Marchioro
0.1.3	22/04/2026	Giulia Barzon	Correzione e diagramma sezione Facade, rivisto MVVM	Elisa Marchioro
0.1.2	22/04/2026	Elisa Marchioro	Sistemata sezione architettura del backend	Giulia Barzon, Matteo Cuogo
0.1.1	21/04/2026	Giulia Barzon	Struttura documento e sezione MVVM, bozza del Facade	Chiara Grossele, Elisa Marchioro
0.1.0	21/04/2026	Giulia Barzon	Completate sezione 1, 2 e 3	Chiara Grossele, Elisa Marchioro
0.0.10	20/04/2026	Elisa Marchioro	Fix alle tabelle	Giulia Barzon
0.0.9	20/04/2026	Giulia Barzon	Mappatura requisiti soddisfatti (una parte)	Elisa Marchioro
0.0.8	18/04/2026	Joseph Grant	Aggiunti i hook frontend	Elisa Marchioro
0.0.7	18/04/2026	Matteo Cuogo	Aggiunto operation mapper	Giulia Barzon, Elisa Marchioro
0.0.6	17/04/2026	Matteo Cuogo	Aggiunta sezione Design Pattern	Giulia Barzon, Elisa Marchioro
0.0.5	17/04/2026	Chiara Grossele	Aggiunta sezione Tracciamento dei requisiti	Matteo Cuogo, Elisa Marchioro
0.0.4	17/04/2026	Joseph Grant	Aggiunta descrizione delle componenti frontend	Chiara Grossele, Matteo Cuogo
0.0.3	15/04/2026	Chiara Grossele	Modifica struttura e aggiunte alla sezione Architettura backend	Joseph Grant, Matteo Cuogo, Giulia Barzon
0.0.2	14/04/2026	Joseph Grant	Aggiunta tecnologie e architettura	Chiara Grossele, Giulia Barzon
0.0.1	28/03/2026	Giulia Barzon	Creazione del documento, scrittura introduzione	Tommaso Tombacco

Tabella 1: Registro delle versioni del documento

Indice

Elenco delle Figure	4
1 Introduzione	1
1.1 Scopo del documento	1
1.2 Approccio alla redazione	1
1.3 Scopo del prodotto	1
1.4 Glossario _G	1
1.5 Riferimenti	1
1.5.1 Riferimenti informativi	2
2 Tecnologie	3
2.1 Framework frontend	3
2.2 Librerie frontend	3
2.3 Linguaggio frontend	3
2.4 Tecnologie di build e runtime	4
2.5 Tecnologie di test	4
2.6 Framework API _G backend	4
2.7 Linguaggio backend	4
2.8 Librerie backend	5
2.9 Server Web	5
2.10 Tecnologie deployment	5
3 Architettura di deployment	6
3.1 Strategia di containerizzazione	6
3.2 Backend (Flask _G)	6
3.3 Frontend (React _G + Nginx _G)	6
3.4 Orchestrazione: Docker Compose _G	6
4 API_G	8
4.1 Principi e requisiti delle API _G REST	8
4.2 Comunicazione client-server (Axios)	8
4.3 Flusso di elaborazione	9
5 Architettura del sistema	10
5.1 Layered Architecture	10
5.2 Applicazione del pattern in "Second Brain"	10
5.2.1 Presentation Layer	10
5.2.2 Application Logic	10
5.2.3 Modulo: <code>app.py</code>	11
5.2.4 Business Logic	11
5.2.5 Modulo: <code>ai_strategies.py</code> (Gestione dei <code>prompt_G</code>)	11
5.2.6 Modulo: <code>operation_mapper.py</code> (Mappatura Operazioni)	11
5.2.7 Modulo: <code>model_strategies.py</code> (Integrazione LLM)	12
5.2.8 Modulo: <code>text_cleaning.py</code> (Servizi di supporto)	12
5.2.9 Persistence Logic	13
6 Backend: design pattern	14
6.1 Strategy pattern	14
6.1.1 Applicazione 1: <code>prompt_G strategies</code> (<code>ai_strategies.py</code>)	15
6.1.2 Applicazione 2: <code>model strategies</code> (<code>model_strategies.py</code>)	17
6.1.3 Caching (<code>get_model</code>)	18

7	Frontend: architettura	19
7.1	Model-View-ViewModel (MVVM)	19
7.2	View: componenti di presentazione	20
7.2.1	Topbar	20
7.2.2	Sidebar	21
7.2.3	EditorButton	21
7.2.4	MarkdownEditor	21
7.2.5	SaveDocumentModal	22
7.2.6	AiPanel	22
7.2.7	AiWidget	22
7.2.8	HistoryList	23
7.2.9	HistoryCard	23
7.3	ViewModel: custom hooks e logica di business	25
7.3.1	useEditor	25
7.3.2	useSaveDocument	26
7.3.3	useFileUpload	26
7.3.4	usePrintDocument	27
7.3.5	useHistory	27
7.3.6	useAiOperations	28
7.3.7	useDraggable	29
7.4	Model: stato e persistenza	30
7.5	Orchestrazione e Dependency Injection	31
7.5.1	EditorPage: orchestratore _G dell'editor	31
7.5.2	HistoryPage: orchestratore _G dello storico	31
7.5.3	Dependency Injection (DI)	31
8	Frontend: design pattern	33
8.1	Facade pattern	33
8.1.1	Applicazione del pattern: gestione della rete	33
8.2	Gestione degli stati del widget AI	36
8.2.1	Problema	36
8.2.2	Soluzione: rendering delegato e stati polimorfi	36
8.2.3	Interfaccia IWidgetState	36
8.2.4	Classi di stato	37
8.2.5	Orchestrazione: store globale e ViewModel	37
8.2.6	Separazione delle responsabilità	38
9	Tracciamento dei requisiti	39
9.1	Notazione dei requisiti	39
9.2	Tracciamento dei requisiti funzionali	39
9.3	Tracciamento dei requisiti di qualità	45
9.4	Tracciamento dei requisiti di vincolo	46

Elenco delle figure

1	Rappresentazione dell'architettura di deployment tramite Docker Compose _G . . .	7
2	Diagramma UML Backend: layered architecture e strategy pattern	14
3	Diagramma UML: Strategy pattern applicato alle operazioni AI	16
4	Diagramma UML: Strategy pattern applicato alla scelta dei LLM	18
5	Diagramma delle classi: architettura globale del frontend secondo il pattern MVVM.	19
6	Diagramma UML dei componenti della View: composizione di EditorPage	20
7	Diagramma UML dei componenti della View: composizione di HistoryPage . . .	23
8	UML UseEditor	25
9	Architettura del modulo History: interazione tra l'hook useHistory e lo store Zustand _G	27
10	UML funzionalità AI: interazione tra ViewModel, Store globale e stati del widget.	29
11	UML del pattern Facade	35
12	Diagramma delle classi del Render-Delegate State Pattern	36

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la **Specifica Tecnica_G** del prodotto *Second Brain*, proposto dall'azienda *Zucchetti* al gruppo ByteMe.

Il documento descrive in dettaglio:

- le **tecnologie** adottate (linguaggi, framework, librerie, strumenti);
- l'**architettura** logica e fisica del sistema, con i pattern architetturali scelti;
- i **design pattern** applicati a livello di componenti, con motivazione delle scelte;
- la **documentazione API_G** tra frontend e backend;
- i **requisiti soddisfatti** con riferimento al documento *Analisi dei Requisiti_G* v2.0.0.

Il documento è rivolto al team di sviluppo come guida implementativa e a committente e proponente come verifica_G della coerenza tra scelte progettuali e requisiti concordati.

1.2 Approccio alla redazione

Il documento viene redatto in modo **incrementale**, assicurando coerenza con gli sviluppi in corso. Non può essere considerato completo fino al rilascio della versione 1.0.0.

1.3 Scopo del prodotto

Il progetto *Second Brain* mira a sviluppare un editor di testo_G avanzato basato sul linguaggio Markdown_G e potenziato dall'integrazione di Large Language Model (LLM). L'applicazione web consentirà all'utente di:

- scrivere e visualizzare in tempo reale testi formattati in Markdown_G;
- interagire con un LLM per riassumere, correggere, riscrivere e tradurre testi;
- ottenere analisi critiche multi-prospettiche secondo il metodo dei *Sei Cappelli per Pensare* di Edward De Bono;
- delegare all'AI la stesura completa di un testo tramite Distant Writing_G.

1.4 Glossario_G

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario_G v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario_G v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

1.5 Riferimenti

- *Norme di Progetto_G* versione 2.0.0
ByteMe/Norme di Progetto_G v2.0.0
- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento del progetto didattico:
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

1.5.1 Riferimenti informativi

- *Analisi dei Requisiti_G* versione 2.0.0 (Ultimo accesso: 20/04/2026)
PB - Analisi dei Requisiti_G v2.0.0
- *Piano di Qualifica* versione 2.0.0 (Ultimo accesso: 20/04/2026)
PB - Piano di Qualifica v2.0.0
- Verbali interni ed esterni
- Glossario_G versione 2.0.0 (Ultimo accesso: 20/04/2026)
Documenti interni - Glossario_G v2.0.0
- Repository_G ufficiale di EasyMDE_G: <https://github.com/Ionaru/easy-markdown-editor>
(Ultimo accesso: 24/03/2026)
- Documentazione linguaggio markdown_G - CommonMark: <https://commonmark.org/>
(Ultimo accesso: 20/02/2026)
- Documentazione React_G 19: <https://react.dev>
(Ultimo accesso: 16/04/2026)
- Principi del Bulletproof React_G (repository_G ufficiale): <https://github.com/alan2207/bulletproof-react>
(Ultimo accesso: 16/04/2026)
- Documentazione Flask_G: <https://flask.palletsprojects.com>
(Ultimo accesso: 10/03/2026)
- Documentazione Docker_G: <https://docs.docker.com>
(Ultimo accesso: 27/02/2026)
- Documentazione TypeScript: <https://www.typescriptlang.org/docs/>
(Ultimo accesso: 27/03/2026)
- Slide del corso di Ingegneria del Software
(Ultimo accesso: 10/04/2026)
- Principi SOLID (slide prof. Cardin)
- Pattern architetturali (slide prof. Cardin)

2 Tecnologie

2.1 Framework frontend

Nome	Versione	Descrizione
React _G	19.2.0	Libreria JavaScript per la costruzione di interfacce utente basate su componenti riutilizzabili.

Tabella 2: Descrizione del framework frontend

2.2 Librerie frontend

Nome	Versione	Descrizione
EasyMDE _G	2.20.0	Editor Markdown _G con anteprima in tempo reale, basato su CodeMirror.
Axios	1.7.9	Client HTTP basato su Promise per effettuare richieste REST dal browser.
React Router _G DOM	7.13.1	Libreria standard per il routing dichiarativo in applicazioni React _G , utilizzata per la navigazione tra le viste.
Zustand _G	5.0.12	Libreria di state management leggera, utilizzata per la gestione dello stato globale e la persistenza dei dati nel LocalStorage.
Lucide-React _G	1.8.0	Libreria di icone vettoriali utilizzata per la UI dell'applicazione.
React _G Hot Toast	2.6.0	Libreria leggera per la visualizzazione di notifiche toast personalizzabili.
CSS Modules _G	N/A (Integrato)	Sistema di scope locale per i fogli di stile CSS integrato in Vite _G , che evita conflitti tra classi.
ESLint	9.39.1	Strumento di linting per TypeScript, che aiuta a mantenere uno stile di codice coerente e a individuare errori.

Tabella 3: Descrizione delle librerie frontend

2.3 Linguaggio frontend

Nome	Versione	Descrizione
TypeScript	5.9.3	Superset tipizzato di JavaScript che aggiunge tipizzazione statica e migliora la manutenibilità del codice.
TSX	N/A (Sintassi)	Estensione di TypeScript che consente di scrivere markup JSX _G all'interno di file tipizzati, usata per i componenti React _G .

Tabella 4: Descrizione dei linguaggi frontend

2.4 Tecnologie di build e runtime

Nome	Versione	Descrizione
Vite _G	7.3.1	Build tool moderno e veloce per applicazioni web, con supporto nativo a ES modules e Hot Module Replacement.
Node.js	20-alpine	Ambiente di runtime JavaScript lato server, utilizzato nel Dockerfile _G per la fase di compilazione del frontend.

Tabella 5: Descrizione delle tecnologie di build

2.5 Tecnologie di test

Nome	Versione	Descrizione
Vitest _G	4.1.4	Framework di testing per applicazioni Vite _G , compatibile con l'API _G di Jest e ottimizzato per TypeScript.
Pytest _G	7.0.0	Framework di testing per Python, con sintassi semplice e supporto a fixture, parametrizzazione e plugin.
React _G testing library	16.3.2	Libreria per il testing di componenti React _G , che fornisce utilities per interagire con i componenti in modo simile a come un utente reale lo farebbe.

Tabella 6: Descrizione delle tecnologie di test

2.6 Framework API_G backend

Nome	Versione	Descrizione
Flask _G	3.1.3	Microframework Python per lo sviluppo di API _G REST, leggero ed estensibile tramite plugin.

Tabella 7: Descrizione del framework API_G backend

2.7 Linguaggio backend

Nome	Versione	Descrizione
Python	3.12-slim	Linguaggio di programmazione ad alto livello, utilizzato per la logica di business e l'implementazione delle API _G .

Tabella 8: Descrizione del linguaggio backend

2.8 Librerie backend

Nome	Versione	Descrizione
OpenAI	2.31.0	Libreria ufficiale utilizzata per l'integrazione e la comunicazione con le API _G dei modelli linguistici (LLM).
Pydantic	2.12.5	Libreria per la validazione _G dei dati e la serializzazione tramite annotazioni di tipo nativo in Python.
Flask _G -CORS	6.0.2	Estensione di Flask _G per la gestione sicura del Cross-Origin Resource Sharing tra frontend e backend.
Python-dotenv	1.2.2	Modulo per il caricamento delle variabili d'ambiente e chiavi API _G dai file <code>.env</code> .
lru-cache	11.3.5	Libreria per implementare una cache in memoria con politica di rimozione LRU (Least Recently Used).

Tabella 9: Descrizione delle librerie backend

2.9 Server Web

Nome	Versione	Descrizione
Nginx _G	alpine	Server web ad alte prestazioni utilizzato nel container Docker _G per servire i file statici del frontend e gestire il routing.
Werkzeug	3.1.8	Server WSGI integrato in Flask _G , utilizzato attualmente come motore di esecuzione per l'ambiente backend.

Tabella 10: Descrizione dei server web frontend e backend

2.10 Tecnologie deployment

Nome	Versione	Descrizione
Docker _G	29.4.0	Piattaforma per la containerizzazione, utilizzata per isolare frontend e backend in ambienti riproducibili.
Docker Compose _G	5.1.2	Strumento per definire e gestire applicazioni multi-container tramite il file <code>docker_G-compose.yml</code> .

Tabella 11: Descrizione delle tecnologie di deployment

3 Architettura di deployment

Il sistema *Second Brain* si basa su un'architettura di deployment di tipo **Client-Server**, distribuita fisicamente su nodi (container) logici distinti. Per l'implementazione pratica, il progetto utilizza la tecnologia di containerizzazione **Docker_G** per garantire l'isolamento dei componenti, la portabilità tra diversi ambienti di sviluppo o produzione e la gestione semplificata delle dipendenze.

3.1 Strategia di containerizzazione

L'applicativo è suddiviso in due servizi principali (il nodo Client e il nodo Server) orchestrati tramite **docker_G-compose**. Questo strumento permette di definire reti virtuali interne, gestire le variabili d'ambiente in modo centralizzato e montare volumi per l'*hot-reloading* durante le fasi di sviluppo.

3.2 Backend (Flask_G)

Il container del backend agisce da Server ed è ottimizzato per l'esecuzione del codice Python in un ambiente leggero e sicuro.

- **Immagine di base:** `python:3.12-slim`. La scelta della variante *slim* riduce l'ingombro dell'immagine base, contenendo solo le librerie di sistema essenziali per il runtime Python 3.12.
- **Configurazione:** La directory di lavoro (`WORKDIR`) è impostata su `/app`. Il sistema installa le dipendenze tramite `pip` utilizzando il flag `--no-cache-dir` per mantenere l'immagine di produzione più snella.
- **Networking e Sicurezza:** Il container espone internamente la porta 8000. Il backend riceve dinamicamente (tramite il file `.env` mappato nel Compose) le chiavi segrete e le credenziali per l'accesso ai provider LLM, evitando di esporre dati sensibili nel codice sorgente.

3.3 Frontend (React_G + Nginx_G)

Il container del frontend agisce da nodo Client. Per la sua realizzazione è stata adottata una strategia di **Multi-stage Build**, che separa l'ambiente di compilazione da quello di esecuzione per minimizzare la dimensione finale dell'immagine.

- **Stage 1 (Build):** Utilizza `node:20-alpine` per installare le dipendenze ed eseguire la build di produzione (`npm run build`) tramite `ViteG`. L'uso di Alpine Linux garantisce velocità nel processo di CI/CD.
- **Stage 2 (Production):** I file statici generati dalla build (`dist`) vengono copiati in un'immagine `nginxG:alpine`. `NginxG` funge da web server statico ad alte prestazioni. Un aspetto cruciale di questo livello è l'iniezione di un file di configurazione personalizzato (`nginxG.conf`), necessario per indirizzare tutte le richieste al file `index.html` e gestire correttamente il routing *client-side* della Single Page Application (SPA).

3.4 Orchestrazione: Docker Compose_G

Il file `dockerG-compose.yml` coordina l'avvio sincronizzato dei servizi e la gestione delle risorse:

- **Mapping delle porte:** Il frontend è reso accessibile all'host esterno sulla porta 8080, mappata sulla porta interna 80 del server `NginxG`.

- **Dipendenze di avvio:** È definita una clausola `depends_on` che assicura che il container del backend sia istanziato prima di quello del frontend. Inoltre, la direttiva `pull_policy: never` protegge l'ambiente di sviluppo dal download accidentale di immagini omonime dai registri pubblici.
- **Ottimizzazione per lo sviluppo (Hot-reloading):** Per velocizzare il ciclo di sviluppo, viene configurato un *bind mount* (`./backend/src:/app/backend/src`) che permette al server Flask_G di riflettere istantaneamente le modifiche al codice sorgente senza dover ricostruire l'immagine del container a ogni salvataggio.
- **Iniezione dell'ambiente:** Il Compose si occupa di orchestrare e iniettare dinamicamente le variabili di configurazione (`environment`), prelevando le chiavi `apiG` dei modelli LLM e i *secret* applicativi direttamente dall'host, mantenendo i container agnostici rispetto alle credenziali.

Architettura di Deployment (Docker Compose) - Second Brain

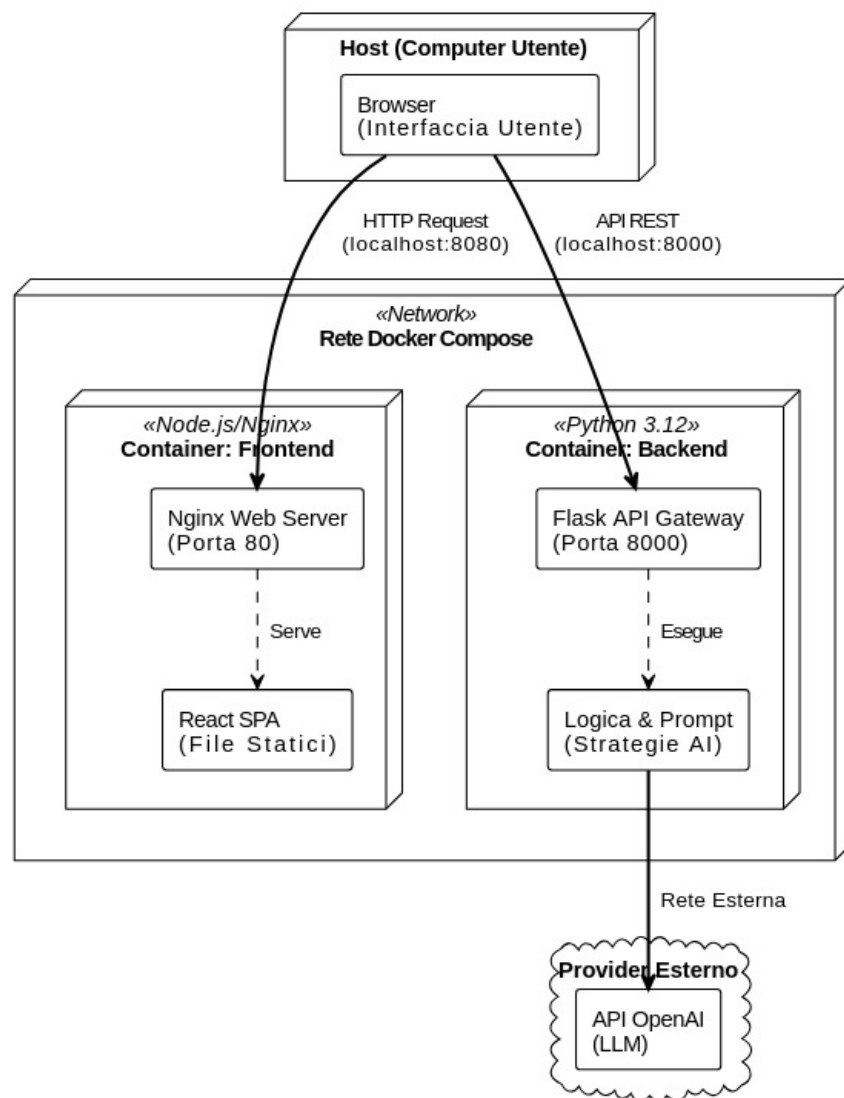


Figura 1: Rappresentazione dell'architettura di deployment tramite Docker Compose_G

4 API_G

Il sistema adotta un'architettura basata su API_G REST (Representational State Transfer), uno stile architetturale per servizi web che consente comunicazione standardizzata, scalabile e disaccoppiata tra client e server. REST si basa sull'utilizzo del protocollo HTTP e sull'identificazione delle risorse tramite URI, promuovendo semplicità, interoperabilità e manutenibilità.

4.1 Principi e requisiti delle API_G REST

Per essere conforme allo stile REST, il sistema rispetta i seguenti principi:

- **Architettura client-server:** separazione netta tra frontend e backend;
- **Statelessness:** ogni richiesta contiene tutte le informazioni necessarie per essere elaborata e il server non mantiene stato tra richieste successive.
- **Interfaccia uniforme:** utilizzo coerente di:
 - URI per identificare le risorse;
 - Metodi HTTP standard: GET → recupero dati; POST → creazione/elaborazione; PUT/PATCH → aggiornamento; DELETE → rimozione.
- **Formato dei dati standardizzato:** scambio dati in formato JSON, facilmente leggibile e interoperabile tra sistemi eterogenei.
- **Separazione delle responsabilità (Separation of Concerns_G):** il client gestisce la presentazione, il server gestisce logica, orchestrazione e integrazione con servizi esterni.
- **Scalabilità:** grazie alla natura stateless_G, il backend può essere replicato e distribuito senza gestione di sessioni condivise.

4.2 Comunicazione client-server (Axios)

Nel frontend, la comunicazione con le API_G REST avviene tramite la libreria **Axios**, un HTTP client basato su promise che semplifica l'invio di richieste asincrone e la gestione delle risposte. Nel contesto dell'applicazione, Axios è utilizzato nel frontend come client HTTP centralizzato per invocare le API_G REST esposte dal backend. In particolare, gestisce l'interazione con l'endpoint_G di generazione AI (POST /api_G/ai/generate), incapsulando tutta la logica di comunicazione.

Nel dettaglio, Axios svolge le seguenti funzioni:

- **Invio delle richieste al backend:** Costruisce e invia la richiesta POST verso l'endpoint_G /api_G/ai/generate contenente:
 - il testo inserito dall'utente (text);
 - l'operazione richiesta (operation);
- **Gestione asincrona della comunicazione:** permette al frontend di utilizzare async/await per gestire la chiamata senza bloccare l'interfaccia utente, migliorando la user experience.
- **Serializzazione automatica dei dati:** converte automaticamente gli oggetti JavaScript in JSON per il body della richiesta, senza necessità di gestione manuale.
- **Gestione della risposta:** riceve il JSON dal backend (es. generated_text) e lo rende immediatamente disponibile al frontend per l'aggiornamento della UI.

- **Gestione degli errori:** centralizza la gestione di errori HTTP o di rete (es. 400, 500), permettendo di definire comportamenti uniformi (messaggi utente, retry, logging).
- **Configurazione condivisa:** consente di definire in un unico punto:
 - baseURL del backend;
 - eventuali headers;
 - timeout e policy di retry.

4.3 Flusso di elaborazione

1. Il frontend invia una richiesta HTTP POST tramite Axios.
2. Il controller backend riceve la richiesta e valida i dati.
3. Viene selezionata la strategia di elaborazione tramite un registry interno.
4. Il sistema costruisce i prompt_G per il modello AI.
5. Il backend invoca il modello per generare il contenuto.
6. La risposta viene pulita e normalizzata.
7. Il risultato viene restituito al client in formato JSON e gestito dal frontend.

5 Architettura del sistema

5.1 Layered Architecture

Il sistema *Second Brain* è stato progettato adottando il pattern architetturale **Layered Architecture** (o Architettura a Livelli), considerato lo standard *de facto* per lo sviluppo di applicazioni moderne e scalabili.

Il principio cardine di questo pattern è la **Separation of Concerns_G** (Separazione delle Responsabilità): i componenti del sistema sono organizzati in strati orizzontali, dove ogni strato fornisce un’astrazione specifica e gestisce esclusivamente la logica di propria competenza. Questo approccio modulare limita il raggio d’azione dei singoli componenti, semplificando notevolmente lo sviluppo, il testing e la manutenzione dell’applicazione.

Vincolo dei "Closed Layers" (Strati Chiusi) Una caratteristica fondamentale di un’architettura a livelli ben progettata è l’isolamento degli strati. Affinché l’architettura sia solida, gli strati devono essere **chiusi**. Questo significa che il flusso di esecuzione segue una direttiva strettamente *top-down*: una richiesta può transitare esclusivamente allo strato immediatamente sottostante. Di conseguenza, i livelli superiori ignorano del tutto i dettagli implementativi dei livelli inferiori, garantendo che le modifiche effettuate all’interno di uno strato non generino impatti a cascata sugli altri.

L’implementazione standard adottata prevede la scomposizione del sistema in quattro livelli logici:

- **Presentation Layer:** Responsabile della gestione dell’interfaccia utente.
- **Application Logic:** Riceve le informazioni dall’esterno, effettua i controlli formali, orchestra le operazioni e gestisce il flusso applicativo.
- **Business Logic:** Il cuore del sistema, che implementa la logica funzionale vera e propria e le regole di dominio, isolato dal sistema di rete.
- **Persistence Logic:** Lo strato dedicato alla conservazione dei dati nel tempo.

5.2 Applicazione del pattern in "Second Brain"

Di seguito viene descritto in dettaglio come i moduli e i file del progetto realizzano i quattro livelli teorici dell’architettura.

5.2.1 Presentation Layer

Questo strato è fisicamente e logicamente disaccoppiato dal backend. È interamente delegato all’applicativo **Frontend** sviluppato in React_G e TypeScript_G. Non esegue elaborazioni semantiche sui testi, ma si occupa esclusivamente di presentare l’interfaccia visiva (UI), raccogliere gli input dell’utente (testo, preferenze di formattazione o di intelligenza artificiale) e inoltrare tali richieste al livello sottostante tramite chiamate api_G HTTP.

5.2.2 Application Logic

L’Application Logic rappresenta il punto di ingresso lato server. Nel sistema, questo ruolo è svolto integralmente dal modulo di routing del framework Flask_G all’interno del file `app.py`, che agisce a tutti gli effetti come **Controller** delle api_G HTTP (comunicanti via JSON). Questo strato non contiene logica di dominio né conoscenza della struttura dei prompt_G o dei modelli LLM: si limita a validare i payload in ingresso e a delegare l’elaborazione agli strati sottostanti, intercettando e formattando eventuali eccezioni.

5.2.3 Modulo: `app.py`

Il modulo non adotta una struttura a classi, ma è organizzato come un insieme di funzioni e configurazioni `FlaskG`.

- `app`: Istanza dell'applicazione `FlaskG`, configurata con CORS abilitato per permettere la comunicazione sicura col frontend.
- `generate_ai_text()`: Funzione principale del modulo, mappata sull'endpoint_G POST `/apiG/ai/generate`. Riceve il payload JSON, estrae i parametri e agisce da *Gateway*: interroga la mappatura delle operazioni, recupera l'istanza del modello e delega alla Business Logic la costruzione dei `promptG`. Infine, restituisce la risposta elaborata al client. In caso di errore, cattura l'eccezione e restituisce una risposta standardizzata con il messaggio di errore.
- `root()`: Funzione mappata sull'endpoint_G GET `/`, utilizzata come *health check* per verificare che il server sia attivo. Restituisce lo stato del servizio e la lista delle operazioni supportate.

5.2.4 Business Logic

La **Business Logic** rappresenta il nucleo funzionale del sistema, completamente isolato dal framework web e dal protocollo HTTP. Questo strato si occupa di orchestrare il flusso dei dati e definire le regole di dominio per la costruzione dei contesti da inviare ai modelli di linguaggio. L'architettura di questo livello è suddivisa in moduli specializzati che utilizzano Design pattern per gestire l'elevata variabilità delle operazioni richieste.

5.2.5 Modulo: `ai_strategies.py` (Gestione dei `promptG`)

Questo modulo contiene la logica di business necessaria per la generazione dinamica dei `promptG`. Definisce i parametri operativi, come la temperatura_G, e le istruzioni esatte per ogni tipo di elaborazione richiesta (editoriale, traduzione o analisi con i Sei Cappelli di De Bono).

Per gestire l'ampia gamma di operazioni in modo estensibile e modulare, le classi contenute in questo file (come `SimplePromptStrategy` e `DeBonoHatStrategy`) implementano il **Pattern Strategy**. Questo permette di isolare la logica di costruzione del testo di sistema dal resto dell'applicazione.

Nota: I dettagli delle singole classi sono approfonditi nella sezione relativa ai Design Pattern.

5.2.6 Modulo: `operation_mapper.py` (Mappatura Operazioni)

Questo modulo funge da punto di giunzione dell'intera architettura, orchestrando il legame tra le strategie di contenuto (`promptG`) e le strategie di esecuzione (modelli LLM).

Problema L'integrazione tra le diverse strategie di prompting e i diversi LLM poneva un problema di accoppiamento rigido e di complessità gestionale. Senza un punto di coordinamento centrale, il Controller (`app.py`) avrebbe dovuto:

- Conoscere l'esistenza di tutte le classi concrete e importarle direttamente.
- Gestire manualmente la logica di accoppiamento tra `promptG` e modelli.
- Contenere lunghi blocchi condizionali per istanziare la strategia corretta in base alla stringa ricevuta dal frontend.

Questa struttura avrebbe violato il principio di Inversione delle Dipendenze, rendendo l'aggiunta di una nuova operazione un processo rischioso e richiedendo la modifica del codice di routing principale ogni volta che viene introdotta una nuova funzionalità. Per ovviare a questo, si è optato per una mappatura centralizzata.

OperationConfig La struttura `OperationConfig` è definita tramite un `@dataclass` ed è progettata per garantire un accoppiamento debole tra i componenti. Agisce come un contenitore che incapsula le dipendenze necessarie a ogni singola operazione.

- **Attributi:**

- `model_class` (`Type[AIModelStrategy]`): è il riferimento al tipo della classe del modello da istanziare, la cui creazione effettiva è delegata al sistema di caching (`get_model`).
- `prompt_g_strategy` (`BaseAIStrategy`): è un'istanza preconfigurata della strategia di `prompt_G` specifica per quell'operazione.

OPERATION_MAPPER Il nucleo del sistema è implementato come un dizionario globale che associa identificatori testuali univoci (es. "summary", "green_hat") a oggetti `OperationConfig`. Questa architettura garantisce tre proprietà fondamentali:

- **Adesione al principio Open-Closed:** Per aggiungere una nuova funzionalità è sufficiente registrare una nuova riga nel dizionario di configurazione. Il codice del controller rimane del tutto inalterato, chiuso alle modifiche ma aperto alle estensioni.
- **Routing dinamico dei modelli:** Permette di assegnare il modello più adatto a una specifica operazione. Ad esempio, nel sistema le operazioni editoriali e di traduzione sono instradate nativamente su Llama 3.2_G, mentre le logiche di pensiero laterale dei Sei Cappelli sono delegate a Gemma 3:4B_G.
- **Robustezza e fallback:** In caso di operazione non riconosciuta o assente nel payload della richiesta, il sistema fornisce una configurazione di default in modo trasparente tramite la costante `DEFAULT_OPERATION = "summary"`, sfruttando il metodo `.get()` del dizionario ed evitando crash dell'applicazione.

5.2.7 Modulo: `model_strategies.py` (Integrazione LLM)

Permette l'astrazione della comunicazione con i provider esterni. Le classi contenute in questo file si occupano di iniettare le chiavi `API_G` lette dall'ambiente e di impacchettare la richiesta nel formato corretto. Anche in questo caso le classi (`ZucchettiLlamaStrategy`, `Gemma3Strategy`) implementano il **Pattern Strategy**.

- **Caching** (`get_model`): All'interno di questo modulo è implementata una funzione di ottimizzazione che gestisce un dizionario globale privato (`_model_instances`). Se l'istanza di connessione a un modello non esiste, la crea; alle chiamate successive restituisce il riferimento in memoria, evitando di aprire connessioni ridondanti.

5.2.8 Modulo: `text_cleaning.py` (Servizi di supporto)

Sottomodulo funzionale che utilizza espressioni regolari (regex) per validare e "ripulire" l'output testuale grezzo generato dagli LLM. Il suo scopo è garantire che il testo restituito al client sia privo di metatesto, preamboli discorsivi o tag markdown_G residui, assicurando la coerenza del dominio.

5.2.9 Persistence Logic

Rappresenta lo strato inferiore deputato alla conservazione dei dati. Coerentemente con la containerizzazione Docker_G, l'architettura del backend è stata progettata per essere totalmente **stateless_G** (priva di stato). Non è presente alcun database_G relazionale lato server: la Persistence Logic per salvare i documenti dell'editor e la cronologia delle generazioni è demandata interamente allo storage locale del client, sfruttando le api_G del **LocalStorage**. Questa architettura garantisce abbattimento del carico server, scalabilità orizzontale e *Privacy by Design*.

6 Backend: design pattern

La progettazione del backend di *Second Brain* si basa sull'applicazione di design pattern comportamentali e creazionali. L'obiettivo primario di questa scelta architetturale è garantire l'estensibilità del sistema: l'aggiunta di nuove operazioni AI o l'integrazione di nuovi modelli LLM deve poter avvenire senza alterare il codice di routing esistente, nel pieno rispetto dei principi SOLID (in particolare l'*Open-Closed Principle* e il *Single Responsibility Principle*).

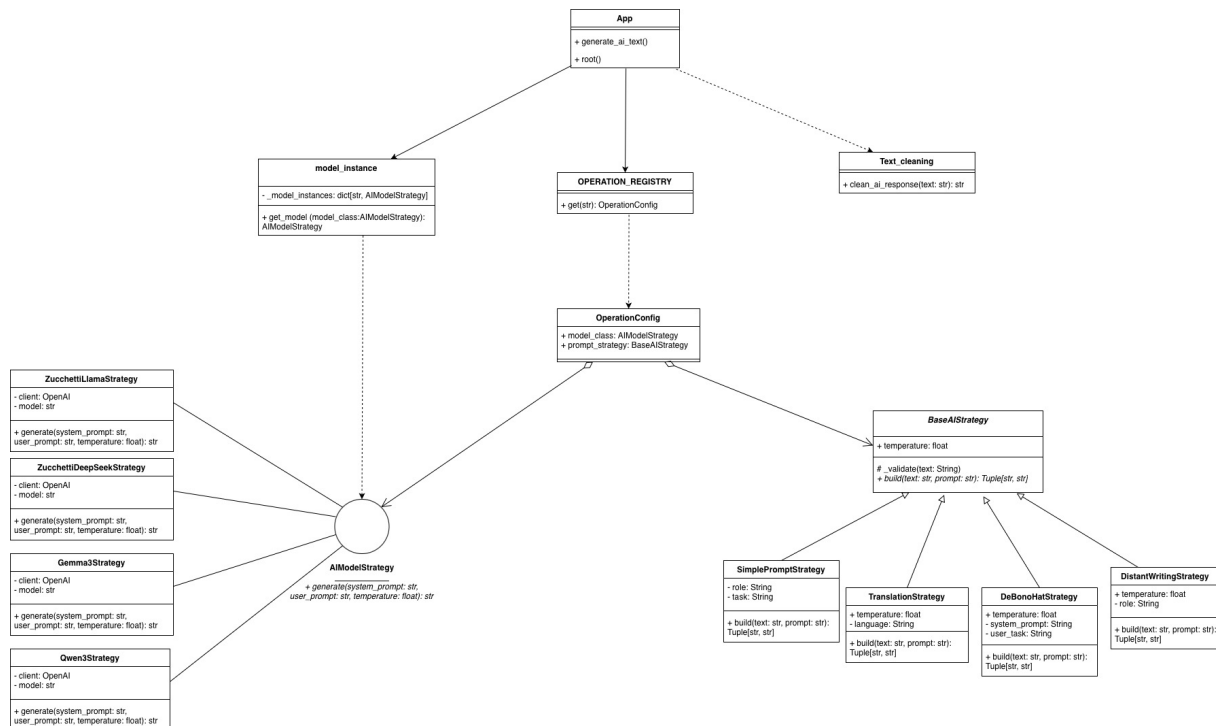


Figura 2: Diagramma UML Backend: layered architecture e strategy pattern

6.1 Strategy pattern

Lo **Strategy** è un design pattern comportamentale il cui scopo principale è definire una famiglia di algoritmi, incapsularli in classi separate e renderli intercambiabili tra loro. Questo approccio permette agli algoritmi di variare in modo indipendente rispetto alle classi che li utilizzano, rendendo il codice modulare ed estensibile.

In pratica, questo pattern si rivela fondamentale quando esistono diverse varianti di una stessa operazione logica e si vuole permettere al sistema di cambiare comportamento a *runtime*, mantenendo inalterata la struttura generale dell'interfaccia.

Struttura Strutturalmente, il pattern si compone di tre elementi chiave:

- **Strategy (interfaccia):** Un contratto generico comune supportato da tutti gli algoritmi della famiglia.
- **ConcreteStrategy:** Le classi concrete che implementano l'interfaccia *Strategy*, definendo i dettagli di uno specifico algoritmo.
- **Context:** L'elemento che necessita dell'algoritmo. Esso mantiene un riferimento all'interfaccia, ignorando i dettagli implementativi della *ConcreteStrategy* con cui è configurato.

Problema L'architettura iniziale del backend presentava un'eccessiva complessità logica dovuta alla gestione di molteplici modelli LLM e diverse logiche di prompting. La presenza di numerosi blocchi condizionali (if/else o switch) all'interno dell'entry point (`app.py`) rendeva il codice rigido, difficile da scalare e violava il principio di singola responsabilità (Single Responsibility Principle).

Vantaggi L'adozione di questo pattern porta tre importanti vantaggi:

1. **Eliminazione degli statement condizionali:** Previene la proliferazione di classi client complesse e di lunghi blocchi if/else o switch per la selezione del comportamento.
2. **Alternativa all'ereditarietà:** Fornisce un approccio basato sulla composizione, più flessibile rispetto all'estensione diretta delle classi.
3. **Adesione ai principi SOLID (Open-Closed):** Permette di aggiungere nuovi algoritmi semplicemente creando una nuova *ConcreteStrategy*, senza modificare il *Context* preesistente.

6.1.1 Applicazione 1: `promptG strategies (ai_strategies.py)`

Questa prima applicazione del pattern gestisce la trasformazione del testo grezzo fornito dall'utente in un input strutturato e ottimizzato per l'IA.

BaseAIStrategy Definisce il contratto astratto comune per tutte le strategie di generazione dei `promptG`.

- **Attributi:**

- **temperature:** Definisce il livello di creatività dell'output. Dichiara un valore di default pari a 0.5, sovrascrivibile nelle classi figlie.

- **Metodi:**

- **_validate():** Metodo concreto ereditabile che verifica_G la solidità dell'input (es. assicura che il testo non sia `None`, sollevando un `TypeError` in caso contrario).
- **build(text: str):** Metodo astratto che impone a ogni sottoclasse di restituire una tupla contenente due stringhe: il System `promptG` (il ruolo dell'IA) e lo User `promptG` (il task specifico).

SimplePromptStrategy Utilizzata per le operazioni editoriali standard (riassunto, correzione grammaticale, riscrittura, distant writing_G). Mantiene la `temperaturaG` di default a 0.5 e costruisce `promptG` diretti impostando istruzioni ferree per evitare preamboli discorsivi.

- **Attributi:**

- **role (str):** Definisce l'identità dell'assistente (es. "Sei un correttore di bozze").
- **task (str):** Definisce l'istruzione specifica da eseguire.

- **Metodi:**

- **build(text):** Combina `role` e `task` con il testo in ingresso per formare la tupla richiesta.

TranslationStrategy Sottoclasse concreta specializzata nella gestione delle attività di traduzione multilingua. Questa classe ottimizza il `promptG` per garantire che l'LLM operi esclusivamente come traduttore professionale, mantenendo la fedeltà linguistica. La `temperaturaG` viene sovrascritta a 0.3 per favorire la massima fedeltà al testo originale.

- **Attributi:**

- `language(str)`: Stringa che definisce la lingua di destinazione (es. "inglese", "cinese mandarino"). Questo attributo viene utilizzato per parametrizzare dinamicamente la richiesta al modello.

- **Metodi:**

- `build(text)`: Implementa la logica di costruzione del `promptG` impostando un `System PromptGG` rigido che definisce il ruolo (traduttore professionista) e i vincoli di output (nessuna frase introduttiva). Lo `User PromptGG` include l'istruzione di traduzione specifica verso la lingua memorizzata nell'attributo `language`.

DeBonoHatStrategy Sottoclasse concreta progettata per implementare il metodo dei "Sei Cappelli per Pensare" di Edward de Bono. Questa strategia guida l'LLM verso un'analisi laterale e strutturata, forzando il modello a osservare il testo da una singola prospettiva cognitiva alla volta (emotiva, logica, creativa, ecc.).

- **Attributi:**

- `hat_name(str)`: Memorizza il nome o il colore del cappello. Serve a definire l'identità dell'IA.
- `focus(str)`: Memorizza l'area di analisi specifica. Serve a guidare il focus del modello durante la generazione.

- **Metodi:**

- `build(self, text)`: Utilizza le f-string per iniettare i valori di `hat_name` e `focus` all'interno dei `promptG`.

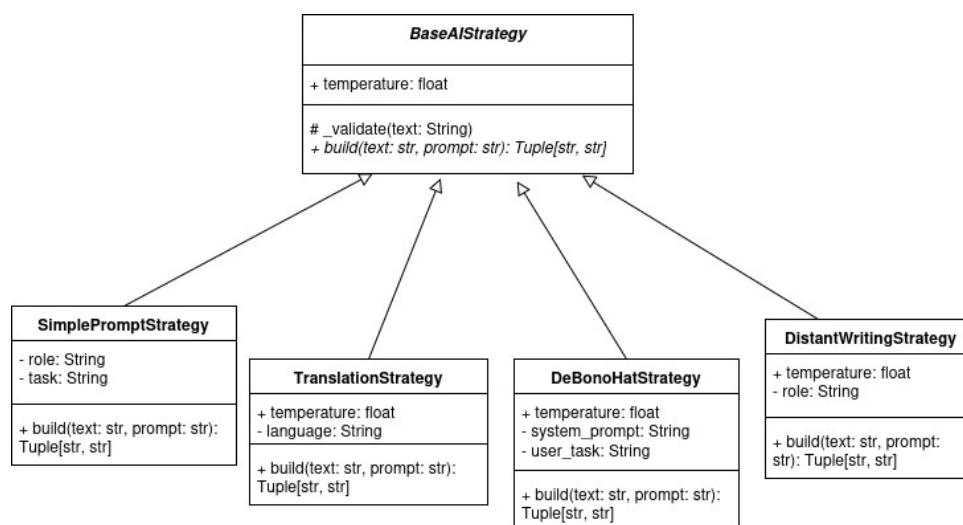


Figura 3: Diagramma UML: Strategy pattern applicato alle operazioni AI

6.1.2 Applicazione 2: model strategies (model_strategies.py)

Questa seconda applicazione del pattern isola completamente i dettagli tecnici (Endpoint_G, API_G Keys, configurazione del client) necessari per comunicare con i diversi provider di Intelligenza Artificiale.

AIModelStrategy Interfaccia comune per tutti i modelli linguistici IA.

- **Metodi:**

- `generate(self, system_prompt, user_prompt, temperature)`: Definisce il contratto per l'invocazione del modello, imponendo alle classi concrete di inviare i prompt_G al fornitore e restituire la risposta come stringa pulita.

Implementazioni concrete (ZucchettiLlamaStrategy e Gemma3Strategy) Il sistema prevede due implementazioni concrete che espongono in modo polimorfo i modelli Llama 3.2_G (per operazioni standard e traduzioni) e Gemma 3_G (per il pensiero laterale dei Sei Cappelli).

- **Attributi:**

- `client`: Istanza del client OpenAI-compatibile, inizializzata con API_G key e base URL letti dalle variabili d'ambiente.

- **Metodi:**

- `model (str)`: Stringa identificativa del modello instradato (es. "llama3.2:3b" o "gemma3:4b").
- `generate(...)`: Implementa la logica di chiamata API_G, inviando i messaggi nel formato richiesto dal protocollo *ChatCompletion*. Estrae e restituisce esclusivamente il contenuto testuale, isolandolo dai metadati di rete (es. token consumati).

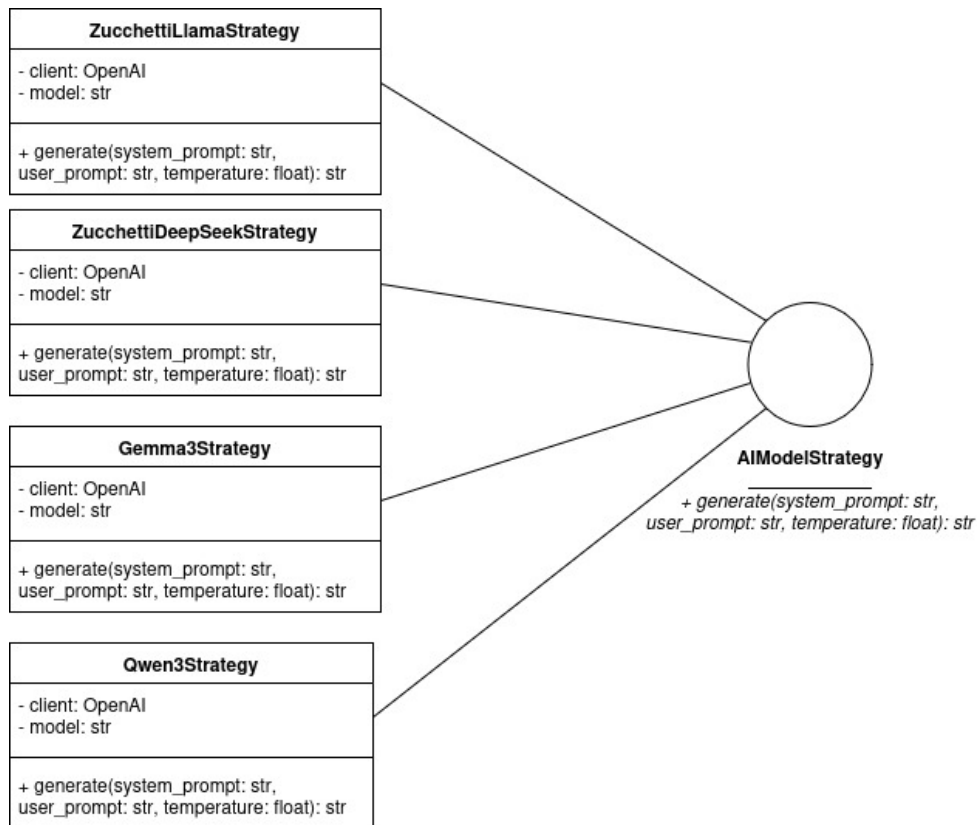


Figura 4: Diagramma UML: Strategy pattern applicato alla scelta dei LLM

6.1.3 Caching (get_model)

La funzione `get_model(model_class)` fornisce un'istanza unica per ciascun modello richiesto, evitando la creazione di più istanze della stessa classe e ottimizzando così l'utilizzo delle risorse.

Il meccanismo di caching è implementato tramite il decoratore `@lru_cache` della libreria standard `functools`. Alla prima richiesta di un determinato modello, l'istanza viene creata e salvata; nelle chiamate successive, viene restituito direttamente il riferimento già memorizzato.

L'utilizzo di `functools` consente di affidarsi a una soluzione robusta e consolidata, che gestisce in modo trasparente anche gli aspetti legati alla concorrenza tra thread. Questo approccio riduce la complessità del codice e migliora l'affidabilità complessiva del sistema.

Nel contesto specifico dell'applicazione, la presenza di più istanze non avrebbe comportato criticità rilevanti, ma si è comunque scelto di adottare una soluzione più rigorosa per garantire maggiore solidità e aderenza alle buone pratiche di progettazione.

7 Frontend: architettura

Il frontend del sistema è stato sviluppato utilizzando **React_G 19**, adottando il pattern architetturale **MVVM** (Model-View-ViewModel). Questa scelta permette di separare nettamente la logica di business dalla logica di presentazione, facilitando il testing isolato dei componenti e la manutenibilità del codice.

L'organizzazione del progetto segue i principi della *Feature-based Architecture_G* (ispirata allo standard di settore **Bulletproof React_G**). Invece di raggruppare i file per tipo (tutti i componenti insieme, tutti gli hook insieme), ogni funzionalità del sistema (es. **editor**, **history**) è isolata in un modulo autocontenuto che include i propri componenti, hook, store e definizioni di tipo.

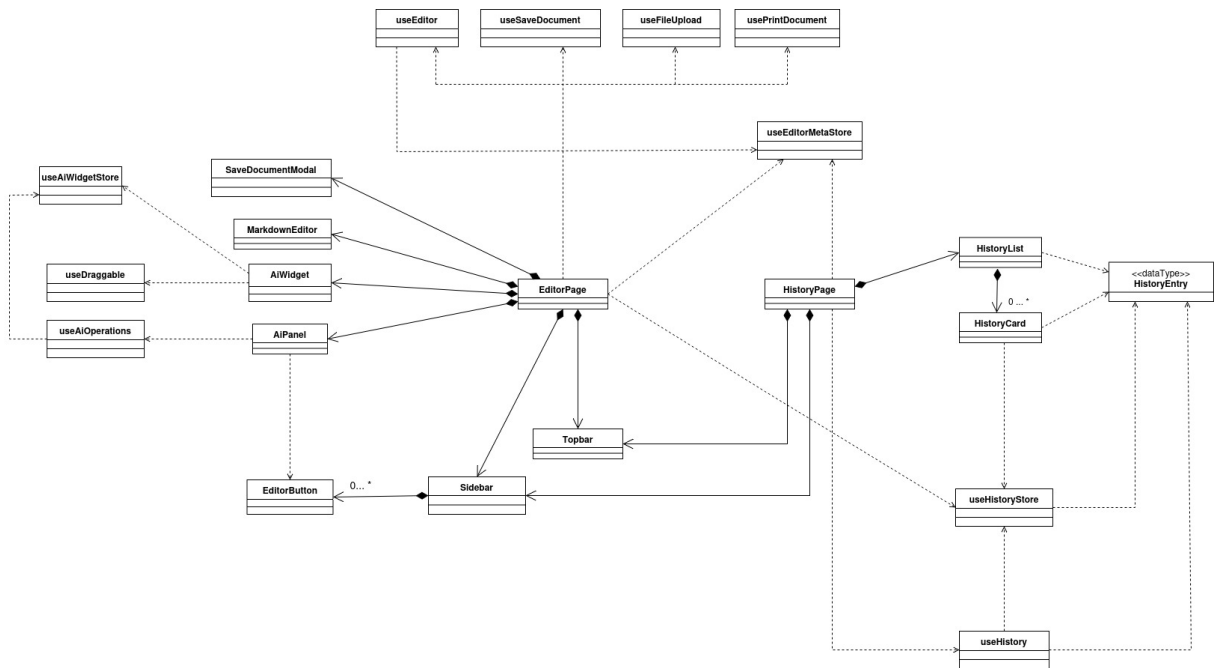


Figura 5: Diagramma delle classi: architettura globale del frontend secondo il pattern MVVM.

7.1 Model-View-ViewModel (MVVM)

Il pattern architetturale **Model-View-ViewModel (MVVM)** è un'evoluzione dei pattern di presentazione (come MVC e MVP) progettata per separare rigorosamente lo sviluppo dell'interfaccia utente (UI) dalla logica di business e dallo stato dell'applicazione. È un pattern di riferimento per lo sviluppo client-side moderno e si adatta perfettamente al paradigma reattivo_G di React_G.

L'architettura si divide in tre strati le cui responsabilità sono nettamente separate:

- **View (vista):** Rappresenta l'interfaccia utente visibile (componenti JSX_G/TSX_G). Nel pattern MVVM, la View è prettamente dichiarativa e "passiva": non detiene la logica di business e non manipola direttamente i dati, ma si limita a renderizzare lo stato che le viene fornito e a delegare le azioni dell'utente ai livelli sottostanti.
- **ViewModel:** Funge da intermediario intelligente tra la View e il Model. È una rappresentazione dello stato e della logica adattata per una specifica vista. Nel nostro sistema è implementato tramite *Custom Hooks*: mantiene lo stato UI temporaneo, espone i dati e i comandi eseguibili, cattura gli eventi generati dalla View (es. click, input) e orchestra le chiamate verso le API_G o verso il Model.

- **Model (modello):** Rappresenta il livello dei dati di dominio e della loro persistenza. Funge da *Single Source of Truth* per l'applicazione. Non ha alcuna conoscenza della View; il suo compito è garantire la consistenza dei dati. Quando lo stato del Model cambia, i ViewModel ad esso sottoscritti (nel nostro caso tramite il pattern *Observer* implementato da Zustand_G) si aggiornano reattivamente, innescando di conseguenza un re-render della View associata.

7.2 View: componenti di presentazione

La View del sistema è realizzata tramite componenti **React_G 19** di tipo funzionale. Per massimizzare la manutenibilità, il gruppo ha adottato un approccio basato su componenti *presentational* (o "dumb"): non detengono logica di dominio, ma si limitano a riflettere lo stato ricevuto dal ViewModel e a propagare le interazioni dell'utente tramite callback. Questo isolamento garantisce che lo strato visivo sia agnostico rispetto alla business logic sottostante.

Di seguito sono descritti i componenti principali che compongono l'interfaccia.

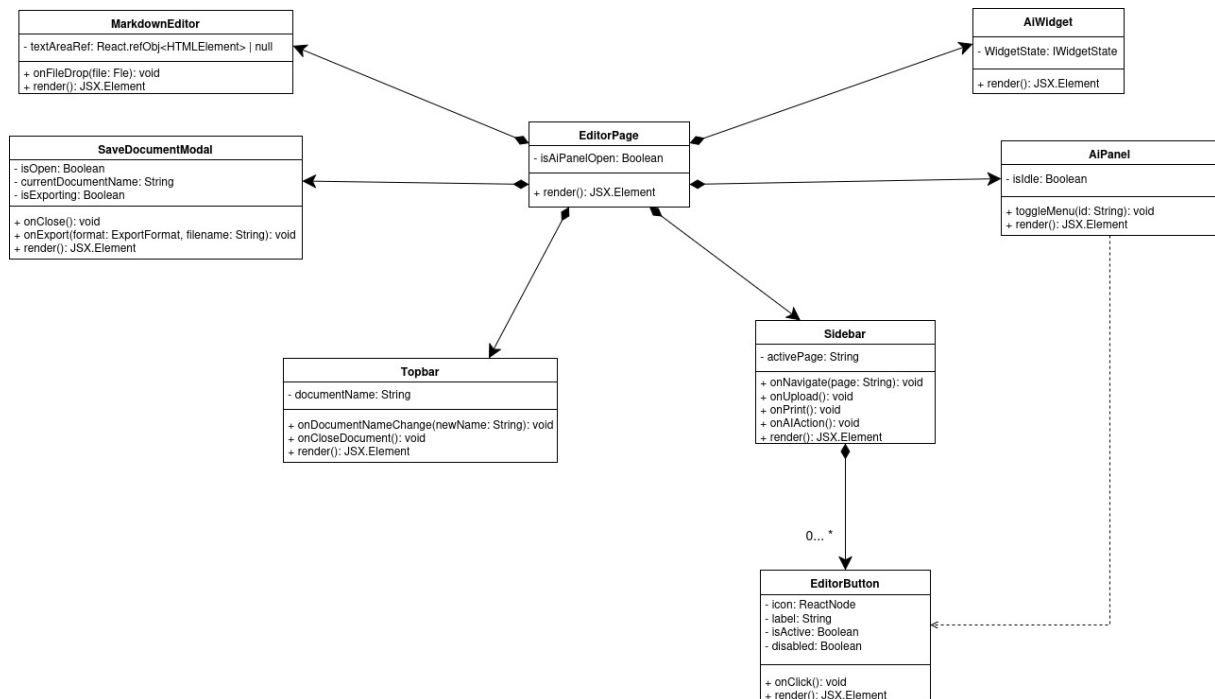


Figura 6: Diagramma UML dei componenti della View: composizione di EditorPage

7.2.1 Topbar

Barra superiore dell'applicazione posizionata in testa all'editor. Il suo scopo principale è visualizzare il nome del documento corrente tramite un campo di input editabile, permettendone la rinominazione rapida.

Props (Dati)

- **documentName:** `string` - nome attuale del documento in fase di editing.

Props (Callback)

- **onDocumentNameChange:** `(newName: string) => void` - funzione invocata alla modifica del titolo.
- **onCloseDocument:** `() => void` - funzione invocata per chiudere il documento corrente.

7.2.2 Sidebar

Barra laterale contenente i pulsanti di navigazione (switch tra editor e storico) e per le funzionalità operative (salvataggio, upload, stampa e azioni AI). Delega l'orchestrazione delle disabilità (es. bloccare i bottoni AI quando si visualizza lo storico) tramite la logica della pagina padre.

Props (Dati)

- `activePage: 'editor' | 'history'` - determina la tipologia di view attualmente attiva per evidenziare il menu corretto (EditorPage o HistoryPage).

Props (Callback)

- `onNavigate: (page: 'editor' | 'history') => void` - aggiorna la view visualizzata.
- `onUpload: () => void` - innesca il caricamento di un file locale.
- `onSave: () => void` - innesca il flusso di salvataggio/esportazione.
- `onPrint: () => void` - invoca la stampa del documento.
- `onAiAction: () => void` - apre il pannello delle azioni di Intelligenza Artificiale.

7.2.3 EditorButton

Componente atomico e riutilizzabile che definisce l'interfaccia standard per tutti i bottoni della Sidebar. Gestisce i propri stati visivi (`isActive`, `disabled`) usando classi generate tramite CSS Modules_G.

Props (Dati)

- `icon: ReactNode` - asset vettoriale/icona del bottone.
- `label: string` - etichetta testuale descrittiva.
- `isActive?: boolean` - determina lo stile di selezione.
- `disabled?: boolean` - inibisce l'interazione e applica lo stile di blocco.

Props (Callback)

- `onClick: () => void` - gestisce l'evento di pressione sul bottone.

7.2.4 MarkdownEditor

Componente delegato al rendering del corpo principale dell'area di scrittura. Consiste in un wrapper reattivo attorno a una `textarea` nativa su cui l'istanza di `EasyMDEG` viene "agganciata" dal ViewModel. Gestisce autonomamente gli eventi di trascinamento (*drag-and-drop*) dei file esterni.

Props (Dati)

- `textAreaRef: ReactG.RefObject<HTMLTextAreaElement | null>` - puntatore fisico al nodo DOM su cui istanziare la libreria esterna.

Props (Callback)

- `onFileDrop?: (file: File) => void` - intercetta il rilascio di un file sull'area di testo.

7.2.5 SaveDocumentModal

Modale interattiva per la configurazione dell'esportazione del documento. Mantiene traccia dello stato locale (formato selezionato, modifiche al nome) e applica tecniche di *sanitizzazione* sul testo immesso prima di confermare.

Props (Dati)

- `isOpen: boolean` - controlla la visibilità a schermo.
- `currentDocumentName: string` - funge da valore predefinito e *placeholder* nell'input text.
- `isExporting?: boolean` - stato transitorio che disabilita i controlli durante l'elaborazione del download.

Props (Callback)

- `onClose: () => void` - innesca la chiusura senza salvataggio.
- `onExport: (format: ExportFormat, filename: string) => void` - passa i parametri sanitizzati al ViewModel per l'esecuzione del salvataggio.

7.2.6 AiPanel

Componente laterale di controllo dedicato alla selezione e all'invocazione delle operazioni di Intelligenza Artificiale. Funge da interfaccia di comando strutturata, organizzando le funzionalità in categorie logiche (operazioni testuali, strumenti di analisi De Bono, traduzioni multilingua).

Responsabilità e logica Essendo un componente connesso, l'`AiPanel` non riceve *props* esterne, ma incapsula le seguenti logiche:

- **Orchestrazione comandi:** Mappa la pressione dei bottoni (`EditorButton`) verso le funzioni esposte dal ViewModel `useAiOperations`.
- **Gestione menù annidati:** Gestisce localmente lo stato di apertura/chiusura dei sotto-menù (es. la lista delle lingue per la traduzione) tramite un set di identificatori (`openMenus`).
- **Controllo di flusso:** Interroga lo store `useAiWidgetStore` per verificare se il sistema è in stato di attesa (`isIdle`); in caso di generazione già in corso, disabilita globalmente i propri comandi per prevenire richieste concorrenti.

7.2.7 AiWidget

Componente flottante e interattivo delegato alla gestione visuale del ciclo di vita di una richiesta AI. Il widget si sovrappone all'interfaccia principale e guida l'utente attraverso le diverse fasi dell'operazione: dall'input di parametri aggiuntivi (`promptG` personalizzati) fino alla visualizzazione e all'inserimento del risultato nell'editor.

Caratteristiche principali Il componente si distingue per un'alta dinamicità e flessibilità architetturale:

- **Rendering polimorfico:** Non possiede un markup statico unico. Utilizza degli stati per delegare il rendering del proprio corpo (**body**) e delle azioni (**actions**) all'oggetto di stato attualmente attivo nello store (**widgetState**).
- **Esperienza utente:** Grazie all'hook `useDraggable`, il widget può essere spostato liberamente dall'utente all'interno dell'area di lavoro (tramite *drag-and-drop* sull'header), evitando di coprire porzioni di testo rilevanti durante la consultazione dei risultati.
- **Disaccoppiamento:** È totalmente agnostico rispetto all'operazione specifica che lo ha attivato; si limita a riflettere lo stato della "macchina a stati" sottostante definita nel ViewModel.

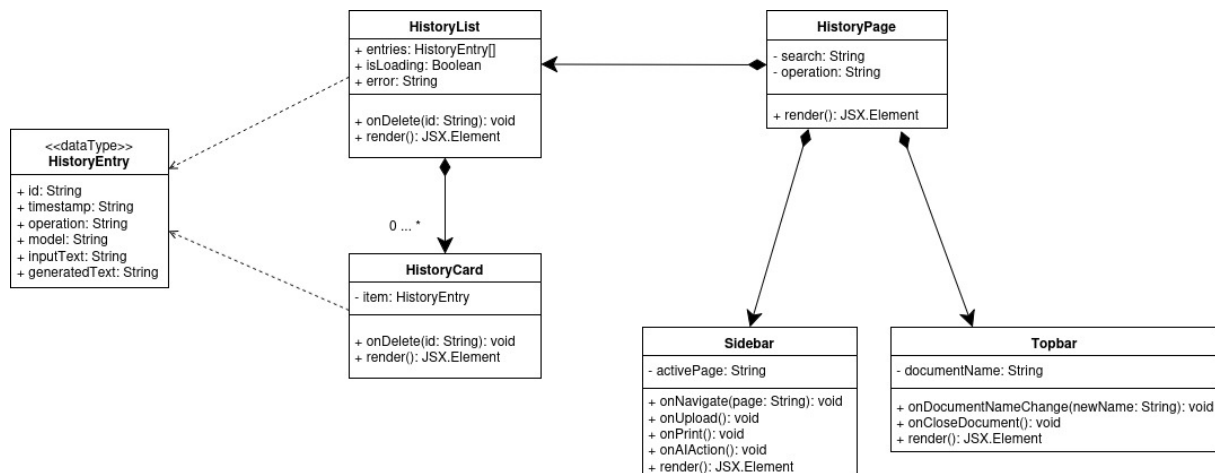


Figura 7: Diagramma UML dei componenti della View: composizione di HistoryPage

7.2.8 HistoryList

Componente per il rendering aggregato dello storico delle generazioni degli LLM. Gestisce internamente l'alternanza visiva tra stati di transizione (caricamento, errore) e lo stato di successo, delegando la visualizzazione del singolo nodo a **HistoryCard**.

Props (Dati)

- **items:** `HistoryItem[]` - array di oggetti rappresentanti le generazioni AI.
- **isLoading:** `boolean` - abilita il rendering dell'indicatore di caricamento.
- **error:** `string | null` - stringa che riporta un messaggio d'errore nel caricamento della cronologia (se presente).

Props (Callback)

- **onDelete:** `(id: string) => void` - propaga verso l'alto la richiesta di rimozione di uno specifico elemento.

7.2.9 HistoryCard

Componente granulare che rappresenta la singola scheda di interazione AI. Incapsula logiche puramente visive tramite `useState`, quali l'espansione/compressione di testi lunghi e il feedback del pulsante di copia negli appunti.

Props (Dati)

- `item: HistoryItem` - struttura dati completa dell'interazione (`promptG`, `output`, `modello` usato, `data`).

Props (Callback)

- `onDelete: (id: string) => void` - gestisce l'eliminazione di una card specifica.

7.3 ViewModel: custom hooks e logica di business

Il ViewModel funge da intermediario tra il Model e la View ed è implementato tramite **Custom Hooks** specializzati. Ogni hook rispetta il *Single Responsibility Principle* (SRP). Il ViewModel ha la responsabilità di:

- Esporre i dati e lo stato degli store alle View in un formato pronto per la visualizzazione.
- Gestire la logica di interazione complessa (ad esempio, l'orchestrazione delle API_G AI tramite la Facade `aiCall`).
- Manipolare direttamente le istanze di librerie esterne, come **EasyMDE_G** per l'editor Markdown_G, isolandone la complessità tecnica all'interno di hook dedicati.

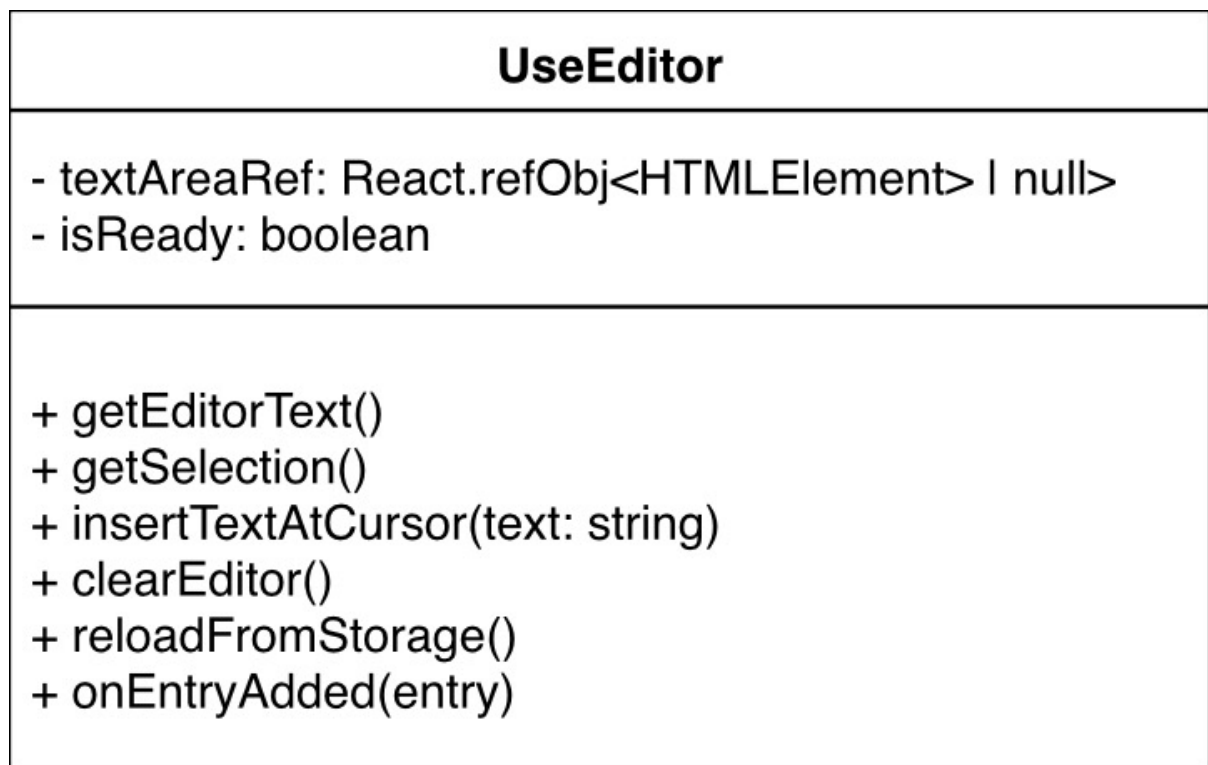


Figura 8: UML UseEditor

7.3.1 useEditor

Hook centrale dell'applicazione. Gestisce il ciclo di vita dell'istanza EasyMDE_G (creazione al mount, distruzione al cleanup). Implementa l'auto-save con tecnica di *debounce* (500 ms) per evitare scritture eccessive su disco. Fornisce all'esterno un'interfaccia solida per interagire con il testo.

Stato/Valori di ritorno

- `textAreaRef`: `ReactG.RefObject<HTMLTextAreaElement | null>` - riferimento iniettato nella view.
- `isReady`: `boolean` - *flag* che indica la completa inizializzazione del motore di rendering Markdown_G.

Funzioni esposte

- `getEditorText: () => string` - restituisce l'intero contenuto testuale corrente.
- `getSelection: () => string` - estrae la porzione di testo evidenziata dall'utente.
- `insertTextAtCursor: (text: string) => void` - inietta testo elaborato (es. output AI) nella posizione attuale del cursore.
- `clearEditor: () => void` - resetta l'area di lavoro.
- `reloadFromStorage: () => void` - forza la sincronizzazione con il LocalStorage.

7.3.2 useSaveDocument

Coordina il flusso di esportazione asincrono. Orchestra in sequenza la composizione sicura del filename, il download fisico (mediato dal file `fileHandler.ts`) e il dispatch verso lo store per aggiornare il timestamp di salvataggio. In caso di errore fallisce silenziosamente mantenendo la modale aperta per un nuovo tentativo.

Stato/Valori di ritorno

- `isOpen: boolean` - stato di visibilità della modale di salvataggio.
- `isExporting: boolean` - stato transitorio che indica se il processo di download è in corso.

Funzioni esposte

- `openModal: () => void` - innesca l'apertura della modale.
- `closeModal: () => void` - innesca la chiusura della modale.
- `handleExport: (rawFilename: string, format: ExportFormat, content: string) => Promise<void>` - processa la richiesta di esportazione, applica la mappa dei MIME type ed esegue il download.

7.3.3 useFileUpload

Gestisce il caricamento asincrono tramite `FileReader`. Esegue controlli preventivi (validazione del formato supportato `.md` o `.txt`, dimensione massima 5 MB) e previene la perdita accidentale di dati non salvati bloccando l'utente e chiedendogli conferma tramite `window.confirm`.

Stato/Valori di ritorno

- `fileInputRef: React.RefObject<HTMLInputElement>` - riferimento fisico all'input nascosto di tipo file nel DOM.

Funzioni esposte

- `triggerFileInput: () => void` - simula il click sull'input nascosto per aprire la finestra di dialogo di sistema.
- `handleFileUpload: (event: React.ChangeEvent<HTMLInputElement>) => Promise<void>` - intercetta il file selezionato, lo valida e ne legge il contenuto testuale.

7.3.4 usePrintDocument

Isola e rende testabile la logica di invocazione della stampa di sistema (`window.print()`). Implementa *guard clauses* per impedire l'avvio del processo se il documento risulta vuoto. Questo hook opera esclusivamente in modo imperativo, non ha quindi nessuno stato locale esposto.

Funzioni esposte

- `handlePrint: (content: string) => void` - verifica_G la lunghezza del contenuto e innesca la stampa del browser o genera un errore visivo.

7.3.5 useHistory

Interfaccia la View con lo store dedicato (`useHistoryStore`), estraendo le interazioni passate. Per garantire alte prestazioni e fluidità UI, tramite `useMemo` applica dinamicamente i filtri di ricerca e l'ordinamento cronologico, assicurando che lo strato di presentazione riceva sempre informazioni formattate senza subire calcoli ridondanti.

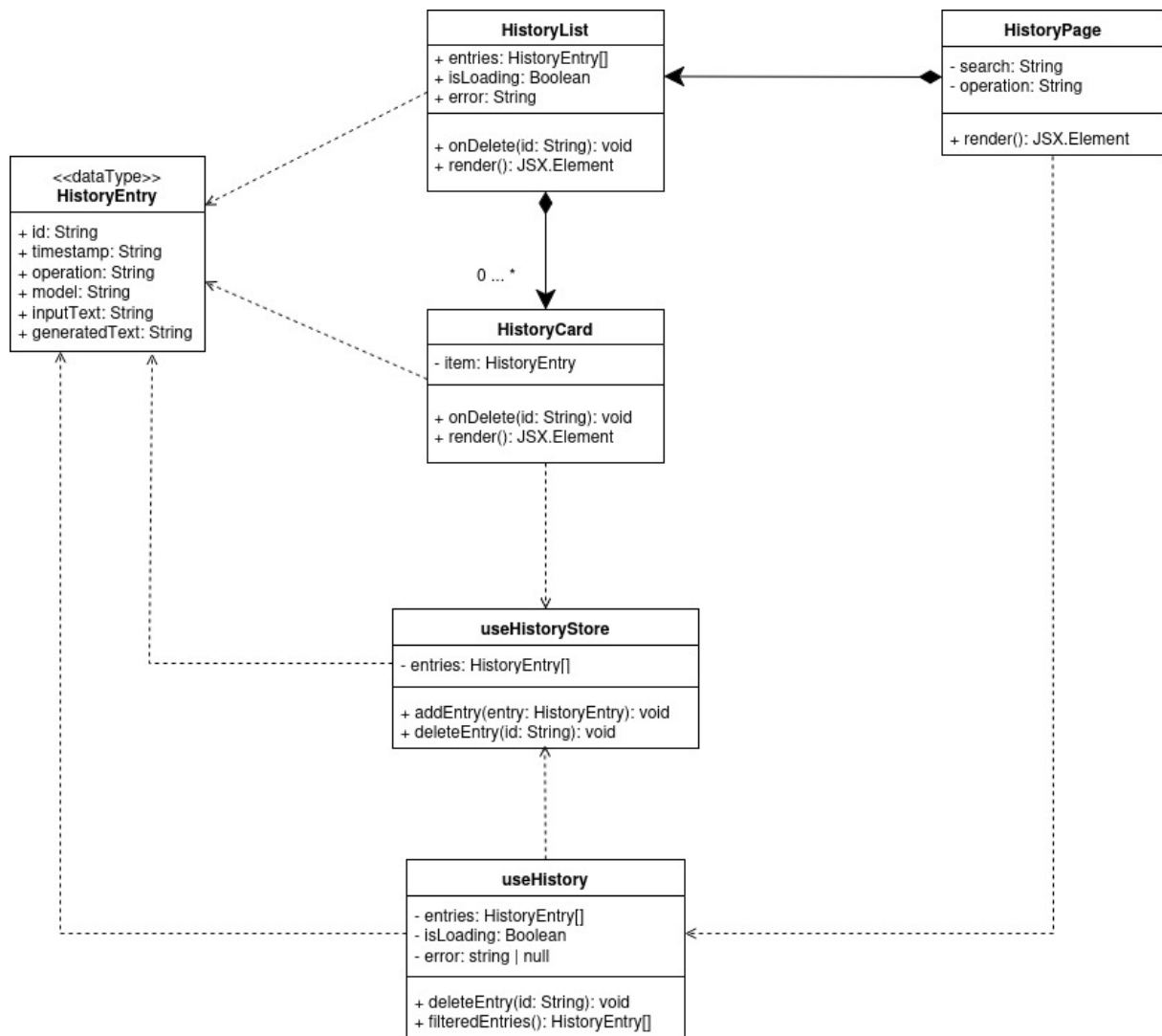


Figura 9: Architettura del modulo History: interazione tra l'hook useHistory e lo store Zustand_G.

Stato/Valori di ritorno

- **entries:** `HistoryEntry[]` - array delle interazioni, eventualmente filtrato e ordinato in base ai criteri correnti.
- **isLoading:** `boolean` - indica se i dati sono ancora in fase di recupero dal `LocalStorage` o dallo store.
- **error:** `string | null` - eventuale stringa di errore occorso durante il fetching della history.

Funzioni esposte

- **deleteEntry:** `(id: string) => void` - innesca la cancellazione di una specifica riga dallo storico persistente.

7.3.6 useAiOperations

Hook orchestratore_G dedicato alla gestione delle interazioni con l'Intelligenza Artificiale. Funge da regista tra la View, i servizi di rete e lo stato globale del widget, isolando la logica di invocazione della Facade `aiCall` e gestendo il ciclo di vita delle richieste asincrone tramite un `AbortController` mantenuto in un `ReactG.RefObject` (`controllerRef`). È l'unico punto del sistema responsabile della creazione delle istanze delle classi di stato, nelle quali inietta le callback operative necessarie, e dell'invocazione di `changeState` sullo store `aiWidgetStore`.

Props (dipendenze) L'hook riceve in ingresso un oggetto di configurazione per interagire con l'editor in modo disaccoppiato:

- **getEditorText:** `() => string` - funzione per recuperare il contenuto totale.
- **getSelection:** `() => string` - funzione per recuperare il testo selezionato.
- **insertText:** `(text: string) => void` - funzione per iniettare il risultato nell'editor.
- **onEntryAdded?:** `(entry: any) => void` - callback opzionale per il salvataggio nello storico.

Gestione dell'annullamento Prima di ogni nuova richiesta, l'hook invoca `abort()` sull'eventuale controller precedente per annullare operazioni in corso, creando poi un nuovo `AbortController`. La funzione interna `handleCancel` incapsula questo comportamento e lo espone agli stati che ne necessitano (es. `LoadingState`).

Funzioni esposte

- **handleAction:** `(operation: AiOperationId) => Promise<void>` - funzione principale che avvia il flusso AI. Intercetta il caso `"distant_writing"` e lo delega a `handleDistantWriting`; per tutte le altre operazioni, esegue direttamente il flusso standard.
- **handleDistantWriting:** `() => void` - funzione che inizializza il flusso con input utente, portando il widget in `InputState` con la callback `onConfirm` preconfigurata.

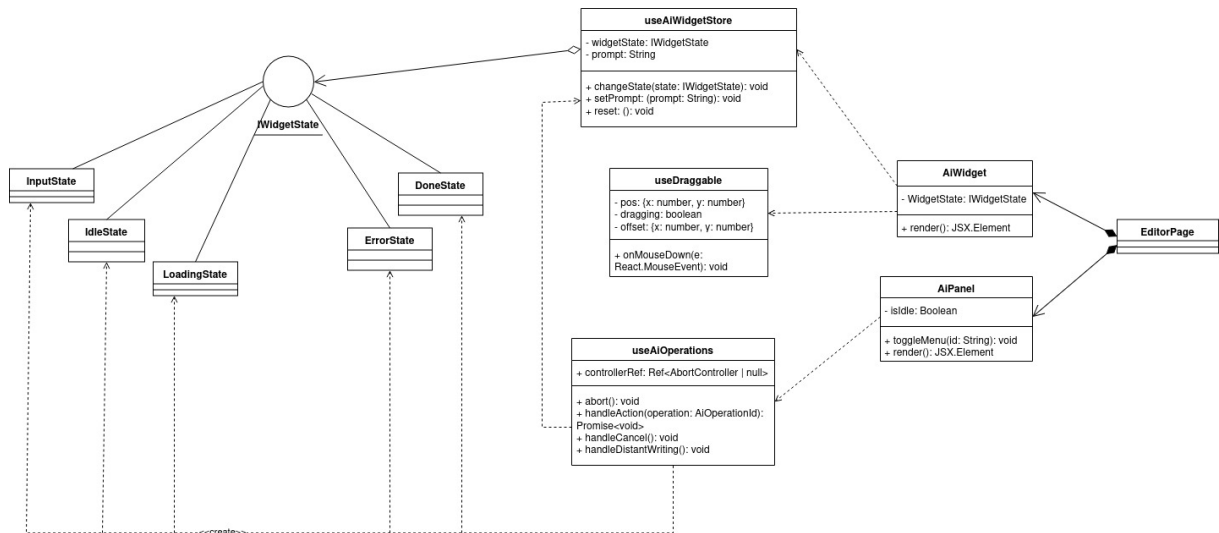


Figura 10: UML funzionalità AI: interazione tra ViewModel, Store globale e stati del widget.

7.3.7 useDraggable

Hook di utilità incaricato di gestire la logica di posizionamento e trascinamento (*drag-and-drop*) di componenti flottanti all'interno dell'interfaccia. Astrae la gestione dei *listener* degli eventi del mouse (`mousemove`, `mouseup`) registrati a livello di `window`, garantendo che il trascinamento funzioni correttamente anche quando il cursore fuoriesce dai limiti del componente durante il movimento rapido.

Scelte implementative L'hook adotta `useRef` per i campi `dragging` e `offset`, anziché `useState`: essendo questi valori aggiornati ad alta frequenza durante il *drag*, evitare re-render inutili è fondamentale per la fluidità visiva. Solo `pos` è dichiarato con `useState`, poiché è il dato che deve propagarsi al componente per aggiornarne il rendering. Il *listener* `onMouseDown` è stabilizzato tramite `useCallback`, evitando che il riferimento alla funzione cambi ad ogni *render* del componente ospitante. Al rilascio del mouse (`mouseup`), i *listener* vengono rimossi tramite `removeEventListener`, prevenendo accumuli di *handler* e potenziali *memory leak*.

Stato/Valori di ritorno

- `pos`: `{ x: number, y: number }` - coordinate spaziali attuali del componente, aggiornate reattivamente durante il trascinamento.

Funzioni esposte

- `onMouseDown`: `(e: React.MouseEvent) => void` - *handler* da collegare all'area sensibile al trascinamento (es. l'header di `AIWidget`). Al momento del click calcola l'offset tra la posizione del cursore e l'origine del componente, poi registra i *listener* a livello di `window` per tracciare il movimento e rilevare il rilascio.

7.4 Model: stato e persistenza

Il Model rappresenta lo stato dei dati e della loro persistenza. Nel sistema non è una struttura passiva, ma è implementato tramite store globali gestiti con **Zustand_G**, che fungono da unica fonte di verità (*Single Source of Truth*) per lo stato dell'applicazione. Sono presenti due store principali:

- **useEditorMetaStore**: Gestisce i metadati dell'editor attivo, ovvero il nome del file (**fileName**), lo stato di modifica non salvata (**isDirty**) e il timestamp dell'ultimo salvataggio (**lastSaved**). Tramite il middleware **persist** di Zustand_G, sincronizza automaticamente questo stato nel **localStorage** del browser, garantendo che i dati sopravvivano al riavvio della pagina.
- **useHistoryStore**: Gestisce l'elenco delle generazioni AI prodotte. Ogni voce è modellata dall'interfaccia **HistoryEntry**. Anche questo store utilizza il **localStorage** per rendere lo storico persistente tra le sessioni.
- **aiWidgetStore**: Gestisce lo stato transitorio e non persistente dell'interfaccia dedicata all'intelligenza artificiale. A differenza degli altri store del sistema, questo modulo non utilizza il middleware di persistenza, garantendo che il widget ritorni sempre alla condizione iniziale di riposo (**IdleState**) ad ogni ricaricamento della pagina. Al suo interno memorizza l'istanza dell'oggetto di stato corrente (**widgetState**), permettendo il rendering polimorfo del widget, e mantiene il testo temporaneo inserito dall'utente per le richieste personalizzate (**prompt_G**).

Nota: Il contenuto testuale effettivo dell'editor, per ragioni di performance legate alla natura della libreria EasyMDE_G, non transita attraverso lo store globale, ma viene salvato e sincronizzato direttamente in una chiave dedicata del **localStorage** gestita dal ViewModel dell'editor.

7.5 Orchestrazione e Dependency Injection

A legare i tre strati architetturali (Model, View, ViewModel) intervengono le **Page** (**EditorPage** e **HistoryPage**). Questi componenti non appartengono né alla View pura (poiché conoscono lo stato globale e gli hook) né al ViewModel (poiché restituiscono markup JSX_G). Essi fungono da **Orchestratori**.

Il ruolo dell'orchestratore_G è quello di istanziare i ViewModel, leggere lo stato dai Model e far fluire i dati e le callback verso il basso (tramite *prop drilling*_G limitato a un solo livello) per distribuirli alle View.

7.5.1 EditorPage: orchestratore_G dell'editor

EditorPage rappresenta l'entry point principale dell'applicazione e coordina la logica complessa dell'editor. Si occupa di:

- **Gestione dello stato locale:** Controlla stati puramente visivi e non condivisi, come l'apertura/chiusura del pannello AI (`isAiPanelOpen`).
- **Collegamento al Model:** Legge e aggiorna i metadati del documento (es. `fileName`, `isDirty`) richiamando `useEditorMetaStore`.
- **Istanziamento del ViewModel:** Monta gli hook dedicati (`useEditor`, `useSaveDocument`, `useFileUpload`, `usePrintDocument`).
- **Composizione degli handler:** Crea funzioni ponte tra la View e il ViewModel. Ad esempio, `handleExportWithContent` recupera prima il testo tramite `editor.getEditorText()` e lo passa al ViewModel di salvataggio; `handleCloseDocument` verifica_G la presenza di modifiche non salvate tramite `window.confirm` prima di resettare l'editor.

7.5.2 HistoryPage: orchestratore_G dello storico

HistoryPage coordina la vista della cronologia delle generazioni AI. Rispetto all'editor, ha un ruolo più limitato:

- **Stato dei filtri:** Mantiene localmente i criteri di ricerca dell'utente (`search` per il testo, `operation` per il tipo di task AI).
- **Delega la logica:** Passa i filtri dinamici al ViewModel `useHistory`, che si occupa del ricalcolo e dell'ordinamento delle voci.
- **Riutilizzo della UI:** Importa la **Topbar** in modalità sola lettura (passando `() => {}` come callback di modifica) e la **Sidebar**, garantendo coerenza visiva con l'editor pur disabilitando i comandi non pertinenti.

7.5.3 Dependency Injection (DI)

La **Dependency Injection** è un pattern architetturale fondamentale dell'Ingegneria del Software, volto a implementare il principio dell'Inversione del Controllo (IoC). Consiste nel fornire a un oggetto o a una funzione le dipendenze di cui ha bisogno dall'esterno, anziché permettere che sia il componente stesso a istanziarle o importarle in modo rigido. Questo approccio riduce l'accoppiamento (*tight coupling*) tra i moduli, aumenta la coesione e rende il sistema altamente modulare e testabile.

Applicazione nel frontend Nel sistema *Second Brain*, questo principio è stato adottato in modo trasversale nell'orchestrazione dei ViewModel. Per evitare dipendenze circolari o un accoppiamento rigido tra domini funzionali diversi, i *custom hook* non importano mai la logica di altri hooks.

Il caso più evidente si trova nell'istanza di `useEditor`: questo hook non importa lo store `useHistoryStore` per salvare le generazioni e ignora del tutto l'esistenza dello storico. Al contrario, è l'orchestratore_G (`EditorPage`) che si fa carico di prelevare l'azione `addEntry` dallo store dello storico per poi iniettarla all'interno di `useEditor` tramite la callback `onEntryAdded`.

Questo approccio garantisce che ogni hook sia modulare e testabile in totale isolamento fornendo dei semplici *mock* al momento dei test.

8 Frontend: design pattern

La progettazione del frontend di *Second Brain* ha fatto uso di design pattern strutturali orientati alla semplificazione dell'interfaccia tra i moduli. L'obiettivo primario, applicando pattern come la Facade, è garantire che la logica di presentazione (View e ViewModel) rimanga agnostica rispetto ai dettagli implementativi della comunicazione di rete, facilitando la testabilità del codice client-side.

8.1 Facade pattern

Il pattern **Facade** è un design pattern strutturale il cui scopo principale è fornire un'interfaccia unica e semplificata per gestire un sottosistema complesso composto da molteplici moduli. L'obiettivo è nascondere le dipendenze e i dettagli implementativi al client, fornendogli un singolo punto di accesso di alto livello dal quale effettuare le richieste. Il Facade riceve i comandi dal client e si occupa di associarli alle logiche specifiche del sottosistema, delegando loro l'effettiva elaborazione. Le classi interne implementano le funzionalità in modo del tutto indipendente, ignare dell'esistenza del Facade stesso.

L'applicazione di questo pattern comporta tre vantaggi architetturali primari:

- **Accoppiamento debole (*Loose coupling*):** Riduce drasticamente la superficie di contatto tra il client e il sottosistema, disaccoppiando chi utilizza i servizi da chi li implementa.
- **Semplificazione e stratificazione:** Diminuisce la complessità generale, elimina le dipendenze circolari e favorisce l'organizzazione del sistema in layer netti e impermeabili.
- **Manutenibilità e testabilità:** Isolare la complessità tecnica in un unico punto centralizzato facilita la scrittura di test automatici (tramite *mocking*) e riduce l'impatto di eventuali refactoring.

8.1.1 Applicazione del pattern: gestione della rete

Nel sistema *Second Brain*, questo pattern è stato applicato per la gestione delle comunicazioni HTTP con il backend. Il sistema implementa una Facade a due livelli sovrapposti, ciascuno con una responsabilità architetturale distinta. I ViewModel e i componenti della View non invocano mai direttamente la libreria esterna *Axios*; al contrario, si interfacciano con moduli dedicati che nascondono la complessità delle transazioni di rete dietro un'interfaccia unificata.

Livello 1: `apiClient.ts` Il primo livello costituisce la Facade sul sottosistema di rete. Non si limita ad avvolgere l'istanza di *Axios*, ma orchestra diverse componenti del networking: la configurazione dell'ambiente, la logica di intercettazione (*Interceptor*), il sistema di formattazione degli errori (`transformApiError`) e il motore di cancellazione (`AbortController`). Fornisce così un'interfaccia unica e semplificata per tutte queste operazioni di basso livello. È responsabile di:

- **Configurazione centralizzata:** Risolve dinamicamente la `baseUrl` dalle variabili d'ambiente (`VITE_G_API_BASE_URL`), con *fallback* a `/api_g`, e imposta il timeout (`VITE_G_API_TIMEOUT`, default 30.000 ms). Garantisce che un unico punto del codice controlli i parametri di connessione.
- **Normalizzazione degli errori:** Tramite un *response interceptor*, tutte le eccezioni di rete (timeout, errori HTTP 4xx/5xx, richieste annullate tramite `AbortSignal`) vengono intercettate e trasformate in oggetti `Error` standard. La logica di normalizzazione è incapsulata nella funzione pura `transformApiError`, testabile in totale isolamento.

- **Gestione della cancellazione:** Il supporto nativo all'`AbortSignal` permette ai View-Model di annullare richieste in corso (es. operazione AI interrotta dall'utente) passando il segnale nella configurazione della chiamata.

L'interfaccia `ApiError` esportata definisce il contratto atteso per le risposte di errore del backend `FlaskG`, garantendo una tipizzazione statica sicura su tutta la catena di chiamate.

Livello 2: `aiCall.ts` (Astrazione dell'endpoint_G AI) Il secondo livello rappresenta una Facade sull'endpoint_G specifico del backend. Nasconde l'URL esatto (`/ai/generate`) e la struttura del payload JSON, esponendo una singola funzione asincrona:

```
executeOperation(payload: AiRequestPayload, signal?: AbortSignal)
  : Promise<string>
```

Il tipo `AiOperationId` enumerato in questo modulo costituisce il contratto tipizzato tra frontend e backend: ogni identificatore (es. `"summary"`, `"white_hat"`) corrisponde a una chiave dell'`OPERATION_MAPPER` lato server, garantendo che il frontend non possa inviare comandi non riconosciuti.

Architettura a cascata e vantaggi I due livelli formano una **Facade a cascata** con responsabilità rigorosamente separate:

- **`apiClient`:** Sa *come* parlare con il server (timeout, header, normalizzazione). Non conosce il dominio applicativo (Intelligenza Artificiale).
- **`aiCall`:** Sa *dove* inviare la richiesta (`/ai/generate`) e *cosa* spedire. Non conosce i dettagli implementativi di `Axios`.
- **`ViewModel` (`useAiOperations`):** Sa *quando* chiamare l'AI e come gestire il risultato nella UI. Non conosce né `Axios` né l'URL dell'endpoint_G.

Questa separazione porta due benefici misurabili in termini di qualità del software:

1. **Disaccoppiamento tecnologico:** L'applicazione è agnostica rispetto al client HTTP. Una futura migrazione all'API_G nativa `fetch` richiederebbe modifiche unicamente al file `apiClient.ts`, lasciando intatti tutti gli hook e i componenti.
2. **Testabilità tramite mocking:** Nei test di unità eseguiti con `VitestG`, isolare la logica di rete è immediato: è sufficiente fornire un *mock* di `executeOperation` senza dover intercettare richieste HTTP reali, rendendo i test deterministici e veloci.

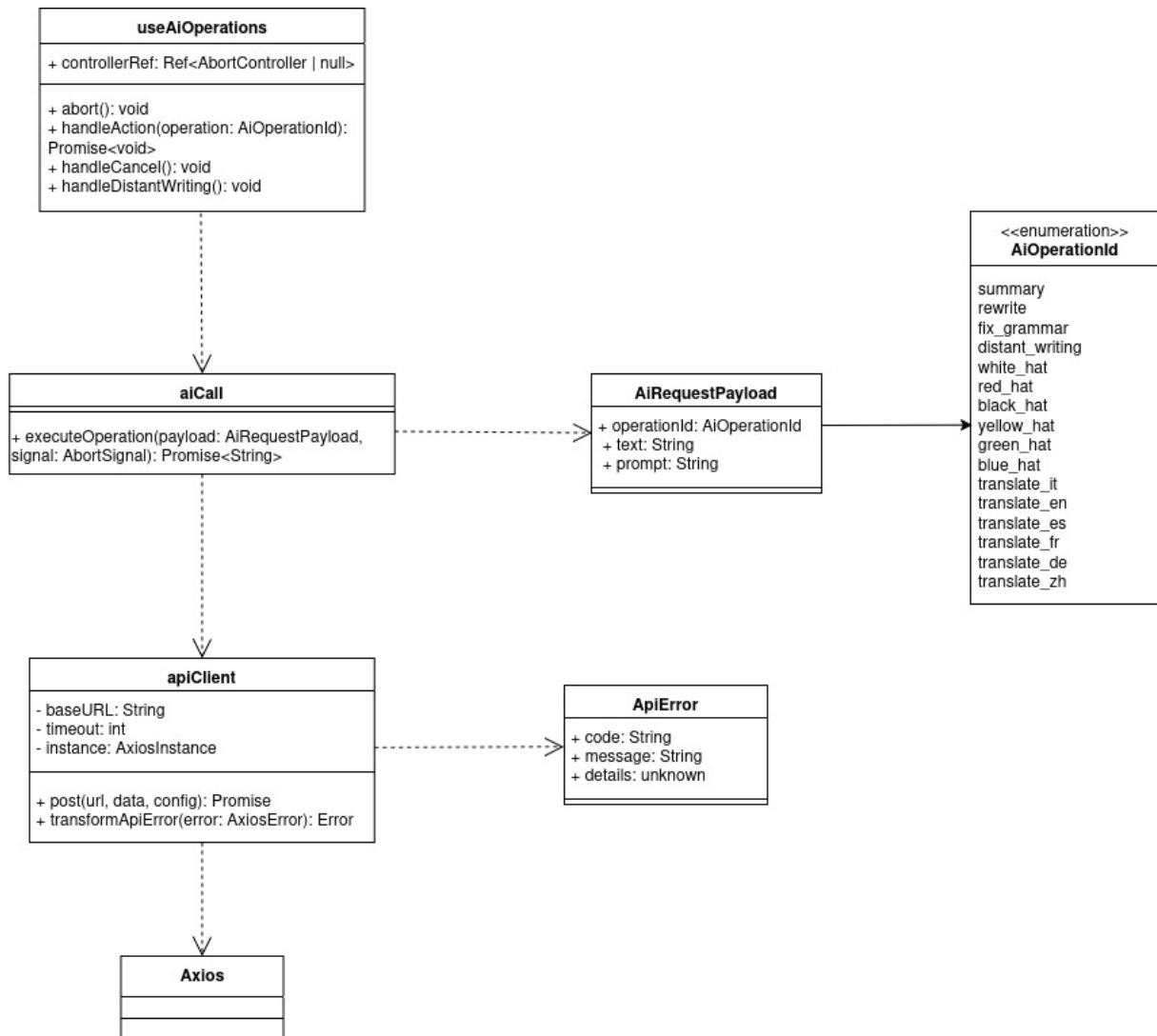


Figura 11: UML del pattern Facade

8.2 Gestione degli stati del widget AI

La gestione dell'interfaccia utente per il widget AI adotta un'implementazione a **stati**. Questa scelta permette di gestire la complessa variabilità visiva del widget in modo modulare e polimorfico.

8.2.1 Problema

L'AI Widget è un componente che espone diverse funzionalità in base allo stato dell'operazione in corso. Ogni operazione attraversa una serie di fasi, rendendo esplicita all'utente la condizione della richiesta. Ad esempio, nel caso di un riassunto, il widget passa da uno stato di caricamento, in cui l'utente attende la risposta e in cui è possibile solo annullare l'operazione, a uno stato di visualizzazione del risultato, che l'utente può inserire, eliminare o rigenerare.

In un ambiente dichiarativo come React_G, adottare questo approccio porterebbe il componente principale a gestire ampi blocchi condizionali (come costrutti **switch** o **if/else**) per determinare quale porzione di interfaccia renderizzare, generando codice rigido, frammentato e difficile da testare.

8.2.2 Soluzione: rendering delegato e stati polimorfi

Nel sistema implementato, il widget si limita a invocare il metodo di rendering dello stato corrente e le relative funzionalità, senza conoscerne l'effettiva implementazione. Ogni stato non è solo comportamentale, ma anche **presentazionale**: ciascun oggetto-stato è direttamente responsabile del proprio rendering in React_G. Questo consente al componente **AiWidget** di agire come un semplice coordinatore passivo.

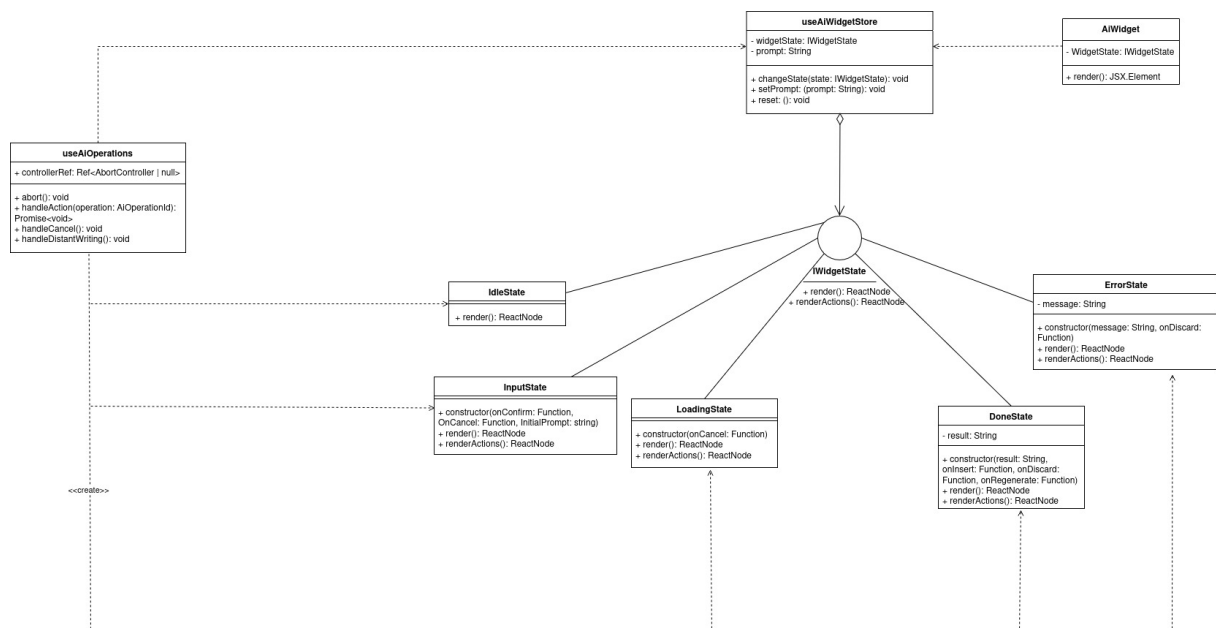


Figura 12: Diagramma delle classi del Render-Delegate State Pattern

8.2.3 Interfaccia IWidgetState

Tutti gli stati implementano l'interfaccia comune **IWidgetState**. Il componente **AiWidget** non dipende da nessuna delle classi concrete, ma conosce esclusivamente i due metodi definiti dal contratto:

- **render()**: Metodo obbligatorio che produce il contenuto visivo principale del widget.

- **renderActions()**: Metodo opzionale che produce i pulsanti contestuali mostrati nel footer dell'interfaccia.

8.2.4 Classi di stato

Sono definite cinque classi concrete, ciascuna corrispondente a una fase del ciclo di vita dell'operazione AI:

- **IdleState**: Stato iniziale e di riposo; l'**AiWidget** non viene renderizzato quando lo stato corrente è un'istanza di questa classe.
- **InputState**: Mostra un'area di testo per l'inserimento del `promptG` da parte dell'utente. Riceve nel costruttore due callback: **onConfirm** (invocata all'invio) e **onCancel** (per l'annullamento).
- **LoadingState**: Visualizza un indicatore di caricamento. Accetta opzionalmente una callback **onCancel** che, se fornita, abilita il pulsante di interruzione della richiesta.
- **DoneState**: Mostra il testo generato e le azioni disponibili: *Inserisci* (inserisce il risultato nell'editor e riporta il widget a **IdleState**), *Scarta* (riporta il widget a **IdleState**) e *Rigenera* che, in base all'operazione riporta a **InputState** con il `promptG` precedente oppure ripete la richiesta con gli stessi parametri.
- **ErrorState**: Visualizza un messaggio d'errore e un pulsante per tornare a **IdleState**.

Le callback iniettate nei costruttori vengono catturate in chiusura al momento della transizione, mantenendo il riferimento al contesto operativo corrente senza che gli stati si conoscano tra loro.

8.2.5 Orchestrazione: store globale e ViewModel

La "macchina a stati" polimorfica è orchestrata attraverso la collaborazione tra lo store globale e la logica di business del ViewModel.

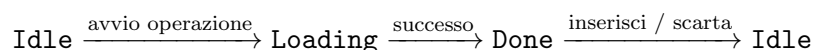
Lo store (aiWidgetStore) Lo stato corrente è mantenuto globalmente in uno store gestito tramite la libreria **Zustand_G**. Nello store, lo stato non è un parametro primitivo, ma un'istanza diretta dell'interfaccia **IWidgetState**. Le transizioni avvengono sostituendo integralmente l'istanza tramite la funzione dedicata **changeState(newState: IWidgetState)**, che innesca l'aggiornamento reattivo dell'interfaccia.

Lo store si occupa inoltre di mantenere il *prompt_G* relativo all'ultima operazione, in modo da poterlo recuperare facilmente in caso di rigenerazione.

ViewModel (useAiOperations) L'orchestrazione delle transizioni è centralizzata nel *custom hook* **useAiOperations**. Questo componente funge da "registra" del pattern:

- Istanza le classi di stato iniettando le callback operative.
- Gestisce le chiamate `apiG` verso il backend.
- Gestisce il ciclo di vita dell'**AbortController** per la cancellazione delle richieste.

Il flusso logico principale gestito dal ViewModel è il seguente:



Nel caso dell'operazione *Distant Writing_G*, che richiede un `promptG` esplicito dell'utente, il flusso viene esteso:

$$\text{Idle} \rightarrow \text{Input} \rightarrow \text{Loading} \rightarrow \text{Done} \xrightarrow{\text{rigenera}} \text{Input}$$

In entrambi i percorsi, un errore durante la chiamata porta istantaneamente a `ErrorState`, mentre la cancellazione esplicita da parte dell'utente riporta direttamente a `IdleState`.

8.2.6 Separazione delle responsabilità

Questa architettura garantisce una netta separazione tra rendering e logica: `AiWidget` è un componente principalmente coordinatore che delega ogni decisione all'oggetto stato corrente; le classi di stato gestiscono la struttura visiva e le callback iniettate, senza contenere logica di business; `useAiOperations` rappresenta l'unico punto in cui risiedono la logica applicativa, le transizioni di stato e la gestione degli errori.

9 Tracciamento dei requisiti

In questa sezione è stato riportato il tracciamento dei requisiti del progetto "Second Brain" con lo scopo di avere una panoramica completa sui requisiti stabiliti nel documento dell'Analisi dei Requisiti_G v.2.0.0 e sul loro conseguimento.

9.1 Notazione dei requisiti

Il codice identificativo segue il formato:

$$R[Importanza] - [Tipo] - [Numero]$$

Dove le voci assumono i seguenti significati:

Importanza

Indica il grado di priorità del requisito:

- **O (Obbligatorio):** Requisito esplicitamente richiesto dall'azienda Zucchetti. La sua mancata implementazione implica il fallimento degli obiettivi primari del progetto.
- **D (Desiderabile):** Requisito non strettamente vincolante ma che il gruppo si impegna ad implementare per migliorare il prodotto.
- **OP (Opzionale):** Requisito di priorità minore, la cui implementazione può essere rimandata.

Tipo

Indica la natura del requisito:

- **F (Funzionale):** Descrive le funzionalità e i servizi che il sistema deve fornire.
- **V (Vincolo):** Descrive limitazioni tecnologiche, normative o implementative imposte esternamente.
- **Q (Qualità):** Descrive le caratteristiche qualitative del sistema (es. performance, sicurezza, manutenibilità).

Numero

Un numero intero progressivo univoco per tipologia di requisito.

9.2 Tracciamento dei requisiti funzionali

Codice	Descrizione	Stato
RO-F-1	Il sistema deve fornire un'area di testo per l'inserimento e la modifica del testo con supporto alla sintassi Markdown _G standard (CommonMark).	Soddisfatto
RO-F-2	Il sistema deve informare l'utente con un messaggio in caso di perdita della connessione Internet durante l'utilizzo dell'editor.	Soddisfatto
RO-F-3	L'editor deve compilare la sintassi Markdown _G in tempo reale.	Soddisfatto
RO-F-4	Deve essere presente un'area di visualizzazione del documento formattato rispetto alla sintassi Markdown _G .	Soddisfatto
RO-F-5	Il sistema deve permettere all'utente di selezionare porzioni di testo tramite mouse o tastiera.	Soddisfatto
RO-F-6	Il sistema deve permettere all'utente di modificare porzioni di testo selezionate.	Soddisfatto

Codice	Descrizione	Stato
RO-F-7	Il sistema deve permettere all'utente di copiare una parte di testo selezionato.	Soddisfatto
RO-F-8	Il sistema deve permettere all'utente di incollare un testo nell'area di testo.	Soddisfatto
RO-F-9	Il sistema deve permettere all'utente di tagliare il testo selezionato.	Soddisfatto
RO-F-10	Il sistema deve permettere all'utente di annullare l'ultima modifica effettuata.	Soddisfatto
RO-F-11	Se non sono presenti operazioni annullabili, il sistema non esegue alcuna azione al comando Undo.	Soddisfatto
RO-F-12	Il sistema deve permettere all'utente di ripristinare le modifiche precedentemente annullate.	Soddisfatto
RO-F-13	Se non sono presenti operazioni ripristinabili, il sistema non esegue alcuna azione al comando Redo.	Soddisfatto
RO-F-14	Nell'editor deve essere presente una barra degli strumenti (toolbar _G) per applicare formattazioni Markdown _G senza scrivere manualmente la sintassi.	Soddisfatto
RO-F-15	Il sistema deve permettere l'applicazione del grassetto al testo tramite toolbar _G .	Soddisfatto
RO-F-16	Il sistema deve permettere l'applicazione del corsivo al testo tramite toolbar _G .	Soddisfatto
RO-F-17	Il sistema deve permettere l'applicazione della sottolineatura al testo tramite toolbar _G .	Soddisfatto
RO-F-18	Il sistema deve permettere di creare intestazioni tramite toolbar _G .	Soddisfatto
RO-F-19	Il sistema deve permettere l'inserimento di elementi di struttura (tabelle, separatori orizzontali) tramite toolbar _G .	Soddisfatto
RO-F-20	Il sistema deve permettere l'inserimento di liste numerate tramite toolbar _G .	Soddisfatto
RO-F-21	Il sistema deve permettere l'inserimento di elenchi puntati tramite toolbar _G .	Soddisfatto
RO-F-22	Il sistema deve permettere l'inserimento di link e riferimenti tramite toolbar _G .	Soddisfatto
RO-F-23	L'utente può applicare più stili contemporaneamente (es. grassetto, corsivo, sottolineatura) alla stessa sezione di testo.	Soddisfatto
RO-F-24	L'utente può annullare una formattazione di una porzione di testo selezionato riapplicandola dalla toolbar _G .	Soddisfatto
RO-F-25	L'utente può estendere la formattazione a una porzione di testo parzialmente formattato riapplicandola dalla toolbar _G .	Soddisfatto
RO-F-26	L'utente può creare sottoliste attivando la creazione di una lista dalla toolbar _G quando il cursore è già all'interno di una lista.	Soddisfatto
RO-F-27	L'utente può scegliere una combinazione di formattazioni dalla toolbar _G quando non è presente testo selezionato, per applicarle al testo che scriverà successivamente.	Soddisfatto
RO-F-28	Il sistema deve permettere all'utente di selezionare porzioni di testo da inviare all'agente esterno.	Soddisfatto
RO-F-29	Il sistema permette di inviare del testo ad un agente esterno.	Soddisfatto
RO-F-30	L'utente deve poter accedere alle azioni sul testo tramite agente esterno in ogni momento.	Soddisfatto
RO-F-31	Il testo generato viene prima mostrato come proposta, separata visivamente dal testo dell'utente.	Soddisfatto

Codice	Descrizione	Stato
RO-F-32	Il testo in stato di proposta include le opzioni di conferma, annullamento e rigenerazione.	Soddisfatto
RO-F-33	Il sistema non permette la modifica del testo generato mentre è in stato di proposta.	Soddisfatto
RO-F-34	Il testo generato dall'agente esterno, una volta confermato, viene inserito nella posizione del cursore se presente, altrimenti in fondo al testo.	Soddisfatto
RO-F-35	Una proposta che viene confermata viene inserita nel documento nell'editor e la sezione di proposta viene rimossa.	Soddisfatto
RO-F-36	Una volta generato e confermato, il testo dell'agente esterno è modificabile tramite le stesse funzionalità dell'editor disponibili per il testo utente.	Soddisfatto
RO-F-37	Una proposta annullata comporta la rimozione del testo generato dalla sezione di proposta e la rimozione della sezione stessa.	Soddisfatto
RO-F-38	Una proposta che viene rigenerata viene rimandata all'agente esterno con gli stessi parametri e la nuova risposta sovrascrive la proposta corrente.	Soddisfatto
RO-F-39	Il sistema deve informare l'utente con un messaggio in caso di errore durante la comunicazione con il servizio LLM.	Soddisfatto
RO-F-40	L'utente può annullare un'operazione AI in corso prima del suo completamento.	Soddisfatto
RO-F-41	In seguito all'annullamento di un'operazione AI, il documento rimane nello stato precedente alla richiesta.	Soddisfatto
RO-F-42	Se la generazione di testo si blocca prematuramente, rimane ciò che è già stato generato, in attesa che l'utente confermi, rigeneri o elimini la proposta.	Soddisfatto
RO-F-43	L'agente esterno applica modifiche esclusivamente al testo selezionato dall'utente. Un contesto aggiuntivo può essere fornito solo a scopo interpretativo e non deve essere alterato.	Soddisfatto
RO-F-44	L'utente può consultare il registro delle generazioni AI effettuate.	Soddisfatto
RO-F-45	Il sistema deve gestire il caso in cui lo storico delle generazioni AI sia vuoto, informando l'utente con un messaggio appropriato.	Soddisfatto
RO-F-46	Ogni generazione di testo dell'agente esterno viene inclusa nel registro delle generazioni, incluse quelle annullate dall'utente.	Soddisfatto
RO-F-47	Il sistema deve permettere all'utente di visualizzare il testo completo di una generazione dallo storico AI.	Soddisfatto
RO-F-48	L'utente deve poter riassumere tutto o una parte selezionata del testo usando un agente esterno.	Soddisfatto
RO-F-49	L'utente deve poter usare un agente esterno per riscrivere tutto o una parte selezionata del testo migliorandolo seguendo determinate specifiche come sintassi, stile e chiarezza.	Soddisfatto
RO-F-50	L'utente deve poter tradurre il testo usando un agente esterno in: inglese, italiano, tedesco, francese, spagnolo e cinese mandarino.	Soddisfatto
RO-F-51	Se l'utente avvia una traduzione senza selezionare la lingua di destinazione, il sistema lo notifica e blocca l'operazione.	Soddisfatto
RO-F-52	Il sistema permette di avviare la correzione grammaticale del testo tramite agente esterno.	Soddisfatto
RO-F-53	Se non vengono rilevati errori grammaticali, il sistema informa l'utente dell'esito positivo della correzione.	Soddisfatto

Codice	Descrizione	Stato
RO-F-54	Il sistema consente all'utente di avviare l'analisi critica del testo selezionato o dell'intero documento utilizzando il cappello di De Bono scelto.	Soddisfatto
RO-F-55	Se il testo è insufficiente per l'analisi critica, il sistema notifica l'utente e interrompe l'operazione.	Soddisfatto
RO-F-56	Se l'utente avvia l'analisi critica senza selezionare un cappello di De Bono, il sistema lo notifica e blocca l'operazione.	Soddisfatto
RO-F-57	Il sistema permette l'invio di un <code>prompt_G</code> scritto dall'utente per la generazione di testo tramite agente esterno.	Soddisfatto
RO-F-58	Il sistema permette la modifica del <code>prompt_G</code> da inviare all'agente esterno prima dell'invio.	Soddisfatto
RO-F-59	L'agente esterno può generare testo basandosi su un <code>prompt_G</code> fornitogli dall'utente.	Soddisfatto
RO-F-60	Se il <code>prompt_G</code> fornito dall'utente è vuoto o incompleto, il sistema notifica l'utente e impedisce l'invio.	Soddisfatto
RO-F-61	L'utente può caricare un file dal proprio dispositivo all'interno dell'editor.	Soddisfatto
RO-F-62	Il sistema deve supportare il caricamento di file in formato <code>Markdown_G</code> (<code>.md</code> , <code>.markdown_G</code>) e testo semplice (<code>.txt</code>).	Soddisfatto
RO-F-63	Il sistema deve supportare il caricamento tramite dialogo di esplorazione file (file picker).	Soddisfatto
RO-F-64	Prima di procedere con il caricamento di un file, il sistema chiede conferma all'utente.	Soddisfatto
RO-F-65	Il sistema deve informare l'utente con un messaggio di esito (successo o errore) al termine di ogni tentativo di caricamento file.	Soddisfatto
RO-F-66	Il sistema deve validare il formato del file selezionato e impedire il caricamento di formati non supportati.	Soddisfatto
RO-F-67	Il sistema deve validare le dimensioni del file selezionato e impedire il caricamento di file che superano la dimensione massima consentita.	Soddisfatto
RO-F-68	Il sistema deve impedire il caricamento di file corrotti o illeggibili.	Soddisfatto
RO-F-69	Il sistema deve permettere all'utente di rimuovere il file che ha caricato.	Soddisfatto
RO-F-70	L'utente può annullare l'operazione di rimozione del file caricato, lasciando l'editor nel suo stato precedente.	Soddisfatto
RO-F-71	Il sistema deve essere in grado di convertire il documento da <code>Markdown_G</code> al formato selezionato dall'utente.	Soddisfatto
RO-F-72	Il sistema deve permettere di scaricare il file nel dispositivo dell'utente.	Soddisfatto
RO-F-73	L'operazione di esportazione o stampa non deve alterare né chiudere il documento originale nell'editor.	Soddisfatto
RO-F-74	Il sistema deve informare l'utente con un messaggio di esito al termine di ogni operazione di esportazione.	Soddisfatto
RO-F-75	Il sistema deve consentire l'esportazione del documento in formato <code>Markdown_G</code> (<code>.md</code>).	Soddisfatto
RO-F-76	Il sistema deve controllare che il range di pagine inserito dall'utente sia valido (non superiore al numero di pagine del documento e non negativo).	Soddisfatto

Codice	Descrizione	Stato
RO-F-77	Il sistema deve permettere di collegare e inviare il documento a un dispositivo di stampa.	Soddisfatto
RO-F-78	Il sistema deve informare l'utente con un messaggio in caso di errore durante l'invio del documento alla stampante.	Soddisfatto
RO-F-79	Il sistema deve garantire la persistenza locale del testo nell'editor tramite browser storage (LocalStorage).	Soddisfatto
RO-F-80	Il sistema deve garantire la persistenza locale dello storico delle generazioni AI tramite browser storage (Zustand _G persist).	Soddisfatto
RO-F-81	Il sistema deve impedire il caricamento di file di dimensioni superiori a 5MB.	Soddisfatto
RD-F-1	Il sistema deve supportare l'uso di scorciatoie da tastiera per le operazioni di formattazione.	Soddisfatto
RD-F-2	Il sistema deve permettere di cambiare modalità di visualizzazione (Solo Editor, Solo Anteprima, Split View).	Soddisfatto
RD-F-3	Il sistema deve supportare il caricamento tramite drag-and-drop (trascinamento file nell'area dell'editor).	Soddisfatto
RD-F-4	Se sono presenti modifiche non salvate, il sistema chiede conferma all'utente prima di caricare un nuovo file.	Soddisfatto
RD-F-5	Il parsing del contenuto del file caricato deve gestire caratteri non validi senza causare il crash dell'applicazione.	Soddisfatto
RD-F-6	I messaggi di errore relativi al caricamento file devono essere esplicativi e suggerire come procedere.	Soddisfatto
RD-F-7	Il sistema deve offrire più opzioni per le operazioni comuni sui file come carica nuovo, stampa, chiudi.	Soddisfatto
RD-F-8	Il sistema deve consentire l'esportazione del documento in formato PDF.	Non soddisfatto
RD-F-9	Il sistema deve consentire l'esportazione del documento in formato HTML.	Soddisfatto
RD-F-10	Il sistema deve consentire l'esportazione del documento in formato ODT.	Non soddisfatto
RD-F-11	Il sistema deve consentire l'esportazione del documento in formato DOCX.	Non soddisfatto
RD-F-12	L'utente deve poter scegliere se esportare l'intero documento o un range di pagine selezionate.	Non soddisfatto
RD-F-13	Il sistema deve consentire all'utente di scegliere il nome del file prima di procedere al download/esportazione.	Soddisfatto
RD-F-14	Il sistema deve fornire un'anteprima del documento nel formato selezionato prima di procedere al download.	Non soddisfatto
RD-F-15	Il sistema deve aprire il dialogo di stampa nativo del browser per configurare le opzioni di stampa.	Soddisfatto
RD-F-16	Il sistema deve informare l'utente con un messaggio se l'editor non risponde ai comandi.	Non soddisfatto
RD-F-17	Il sistema deve informare l'utente con un messaggio se il pannello di anteprima non si aggiorna correttamente.	Non soddisfatto
ROP-F-1	L'utente può ricercare parole nel testo attraverso l'editor	Non soddisfatto
ROP-F-2	Il sistema deve informare l'utente se la ricerca non ha prodotto occorrenze nel testo.	Non soddisfatto
ROP-F-3	La sezione di testo generata come proposta è visibilmente diversa dal testo utente	Soddisfatto

Codice	Descrizione	Stato
ROP-F-4	Per ogni voce del registro delle generazioni AI il sistema deve indicare la data e l'ora della generazione.	Soddisfatto
ROP-F-5	Per ogni voce del registro delle generazioni AI il sistema deve indicare il tipo di operazione che ha prodotto la generazione.	Soddisfatto
ROP-F-6	L'utente può eliminare una voce dal registro delle generazioni AI.	Soddisfatto
ROP-F-7	Per gli utenti autenticati, il registro delle generazioni AI è persistente ed è associato ai documenti salvati nel proprio account.	Non soddisfatto
ROP-F-8	La sezione di testo in stato di proposta viene colorata in base al tipo di operazione che viene fatta	Soddisfatto
ROP-F-9	Il sistema deve visualizzare sia il testo originale che la traduzione generata, permettendo all'utente di confrontarle.	Soddisfatto
ROP-F-10	Se il testo selezionato è già nella lingua di destinazione della traduzione, il sistema avvisa l'utente e interrompe l'operazione.	Non soddisfatto
ROP-F-11	Il sistema offre esempi per la richiesta di generazione di testo a partire da un estratto	Non soddisfatto
ROP-F-12	Il sistema permette di inserire un grafico generato, con dati presenti nel testo, scelto tra diversi tipi disponibili all'utente	Non soddisfatto
ROP-F-13	Il sistema permette di inserire una tabella generata con dati presenti nel testo	Non soddisfatto
ROP-F-14	Se i dati nel testo sono insufficienti per generare grafici o tabelle, il sistema notifica l'utente.	Non soddisfatto
ROP-F-15	Il sistema permette di inserire una mappa concettuale generata con dati presenti nel testo	Non soddisfatto
ROP-F-16	Se si verificano errori nella visualizzazione della mappa concettuale generata, il sistema notifica l'utente.	Non soddisfatto
ROP-F-17	Il sistema permette di inserire una formula matematica generata con dati presenti nel testo	Non soddisfatto
ROP-F-18	Se la formula matematica contiene simboli non supportati, il sistema notifica l'utente.	Non soddisfatto
ROP-F-19	Il sistema deve permettere all'utente di inserire le proprie credenziali (username e password) per il login	Non soddisfatto
ROP-F-20	Se le credenziali sono errate il sistema non permette all'utente di autenticarsi	Non soddisfatto
ROP-F-21	Il sistema deve mostrare un messaggio di errore generico in caso di fallimento del login, senza specificare se l'errore riguarda username o password.	Non soddisfatto
ROP-F-22	In caso di username già esistente durante la registrazione, il sistema deve proporre all'utente di effettuare il login o scegliere un nuovo username.	Non soddisfatto
ROP-F-23	Il sistema deve permettere all'utente di potersi registrare tramite uno username e una password	Non soddisfatto
ROP-F-24	Il sistema deve impedire la registrazione in caso ci sia uno username uguale nel database _G	Non soddisfatto
ROP-F-25	Il sistema deve salvare nel database _G le password criptate in fase di registrazione	Non soddisfatto
ROP-F-26	Il sistema deve permettere ad un utente autenticato _G di poter eliminare il proprio account	Non soddisfatto
ROP-F-27	Il sistema in caso di eliminazione account deve eliminare i dati dal database _G	Non soddisfatto

Codice	Descrizione	Stato
ROP-F-28	Le operazioni di eliminazione dell'account devono richiedere una conferma esplicita da parte dell'utente	Non soddisfatto
ROP-F-29	Il sistema deve terminare la sessione se un account viene eliminato	Non soddisfatto
ROP-F-30	L'utente può annullare la procedura di eliminazione del proprio account.	Non soddisfatto
ROP-F-31	Il sistema deve permettere ad un utente di vedere i file che ha salvato	Non soddisfatto
ROP-F-32	Il sistema deve permettere ad un utente autenticato _G di selezionare un file dall'elenco di quelli salvati e aprirlo	Non soddisfatto
ROP-F-33	Il sistema deve permettere all'utente di annullare l'operazione di apertura file, lasciando l'editor nel suo stato precedente.	Soddisfatto
ROP-F-34	Il sistema deve mostrare i file salvati in ordine cronologico per data di ultima modifica.	Non soddisfatto
ROP-F-35	L'accesso ai file salvati nel database _G è consentito solo ad utenti autenticati.	Non soddisfatto
ROP-F-36	Il sistema deve permettere il salvataggio di un file in un sistema di persistenza _G non locale	Non soddisfatto
ROP-F-37	Il sistema deve notificare l'utente in caso di fallimento del salvataggio nel database _G .	Non soddisfatto
ROP-F-38	Il sistema deve associare ogni file al suo creatore	Non soddisfatto
ROP-F-39	Il sistema deve impedire il salvataggio di file nel database _G da parte di utenti non autenticati.	Non soddisfatto
ROP-F-40	Il sistema deve impedire il salvataggio di un documento vuoto nel database _G .	Non soddisfatto
ROP-F-41	Un utente non autenticato _G che prova a salvare un file viene riportato alla pagina di login	Non soddisfatto
ROP-F-42	L'utente può effettuare il logout in ogni momento e terminare la propria sessione	Non soddisfatto
ROP-F-43	Il sistema deve permettere all'utente di modificare il nome del documento corrente direttamente dall'interfaccia di lavoro.	Soddisfatto
ROP-F-44	Il sistema deve permettere all'utente di cercare testo all'interno del registro delle generazioni AI tramite una barra di ricerca.	Soddisfatto
ROP-F-45	Il sistema deve permettere all'utente di filtrare le voci del registro delle generazioni AI in base al tipo di operazione effettuata.	Soddisfatto

Tabella 12: Tracciamento dei requisiti funzionali_G

9.3 Tracciamento dei requisiti di qualità

Codice	Descrizione	Stato
RO-Q-1	Il tempo di risposta dell'interfaccia utente (aggiornamento anteprima, applicazione formattazioni) deve essere inferiore a 500 ms nel 95% dei casi, misurato su una sessione utente attiva di almeno 30 minuti.	Soddisfatto
RO-Q-2	Il tempo medio di generazione di una risposta da parte del servizio LLM non deve superare i 30 secondi, misurato su un campione di almeno 20 richieste consecutive in condizioni di utilizzo normale.	Soddisfatto

Codice	Descrizione	Stato
RO-Q-3	Il tasso di errori nelle operazioni principali (salvataggio, apertura file, chiamate AI) non deve superare l'1%, misurato sul totale delle operazioni eseguite in una sessione utente attiva di almeno 30 minuti.	Soddisfatto
RO-Q-4	La percentuale di codice coperta dai test automatici (test coverage) deve essere almeno il 70%, misurata sull'insieme completo dei moduli rilasciati ad ogni versione.	Soddisfatto
RO-Q-5	La complessità ciclomatica (McCabe) di ogni funzione/metodo del codice prodotto non deve superare il valore 8.	Soddisfatto
RO-Q-6	La documentazione prodotta deve ottenere un Indice di Gulpease (IG) maggiore o uguale a 40.	Soddisfatto
RO-Q-7	Il sistema deve essere sviluppato rispettando le Norme di Progetto del gruppo ByteMe.	Soddisfatto
RO-Q-8	Il sistema deve essere verificato secondo il Piano di Qualifica prima di ogni rilascio.	Soddisfatto
RO-Q-9	Il manuale utente _G deve documentare tutte le funzionalità obbligatorie del sistema e deve ottenere un Indice di Gulpease (IG) maggiore o uguale a 40.	Soddisfatto
RO-Q-10	Il manuale di installazione e deployment del sistema deve ottenere un Indice di Gulpease (IG) maggiore o uguale a 40.	Soddisfatto

Tabella 13: Tracciamento dei requisiti di qualità_G

9.4 Tracciamento dei requisiti di vincolo

Codice	Descrizione	Stato
RO-V-1	L'applicazione deve essere accessibile tramite browser web. I browser supportati sono: Google Chrome ($\geq$ v110), Mozilla Firefox ($\geq$ v110), Microsoft Edge ($\geq$ v110), Safari ($\geq$ v16).	Soddisfatto
RO-V-2	Il sistema deve utilizzare la codifica Unicode (UTF-8) per la rappresentazione interna e lo storage del testo.	Soddisfatto
RO-V-3	Il formato di input e di storage del testo deve essere Markdown _G , in conformità allo standard CommonMark.	Soddisfatto
RO-V-4	Il sistema deve integrarsi con un servizio LLM tramite API _G REST compatibili con lo standard OpenAI.	Soddisfatto

Tabella 14: Tracciamento dei requisiti di vincolo_G